



Understanding GPU Performance

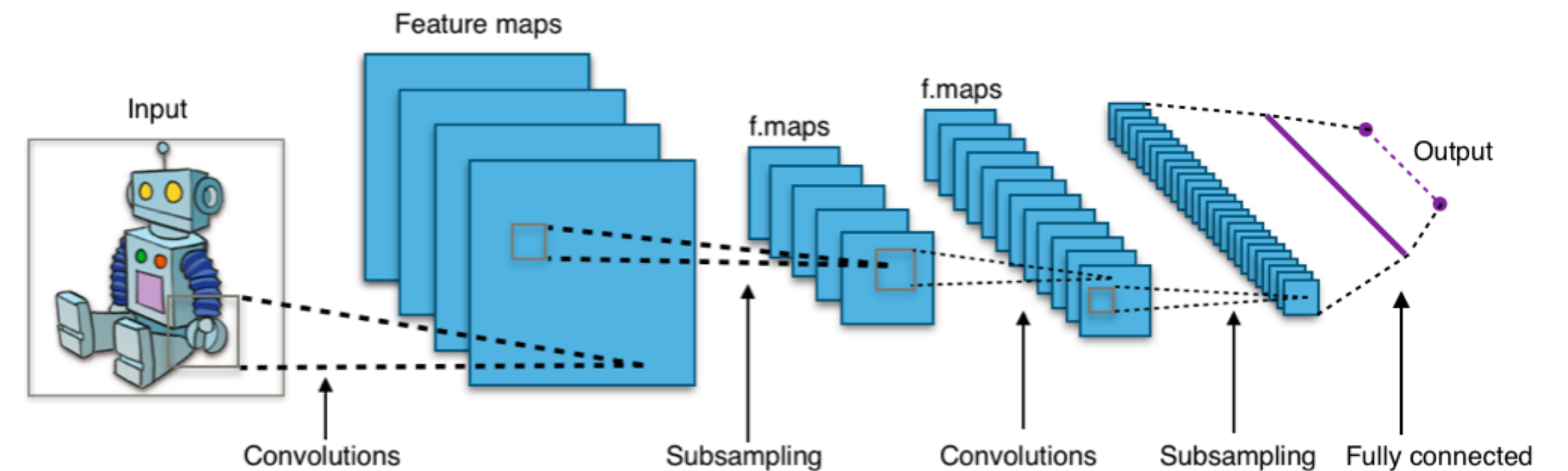
Введение в эффективные системы глубокого обучения

Великонивцев Федор Сергеевич, Yandex Research
Лекция 02, осень 2026

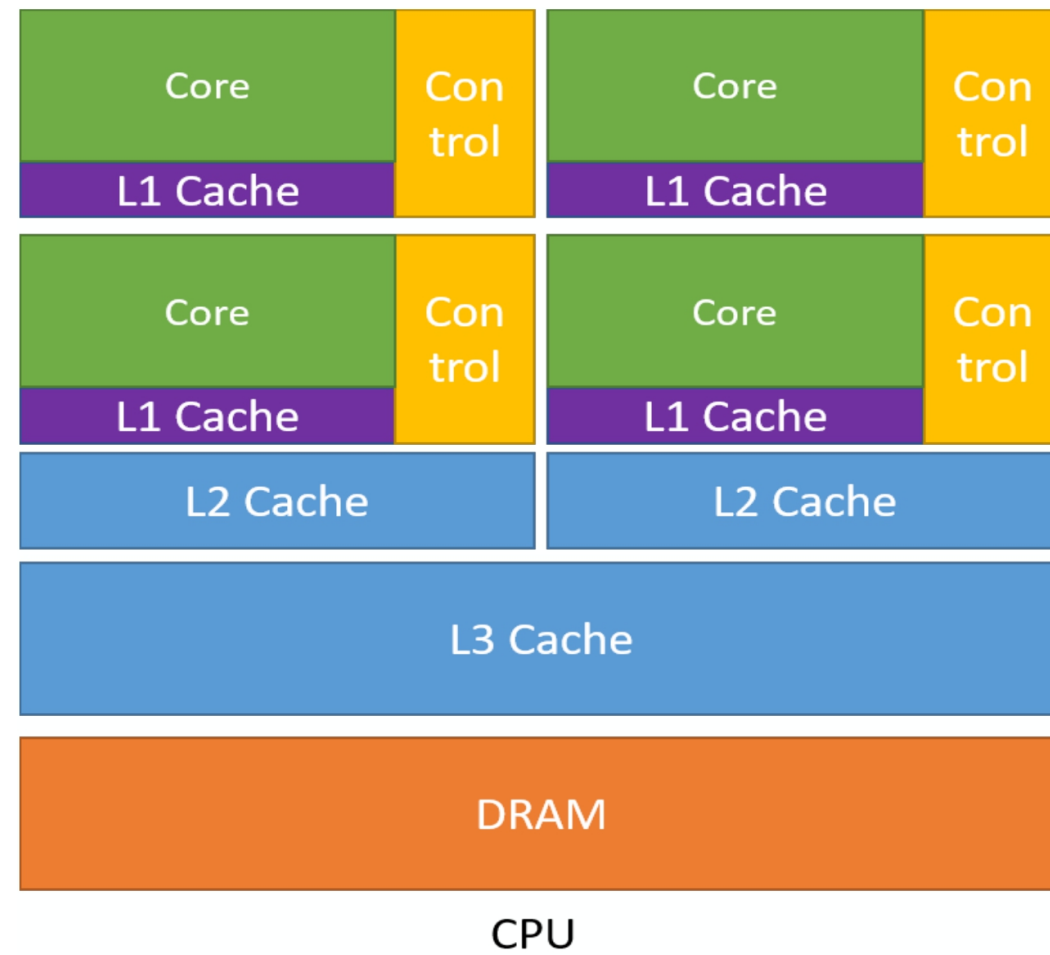
Recap

Why GPUs for Deep Learning?

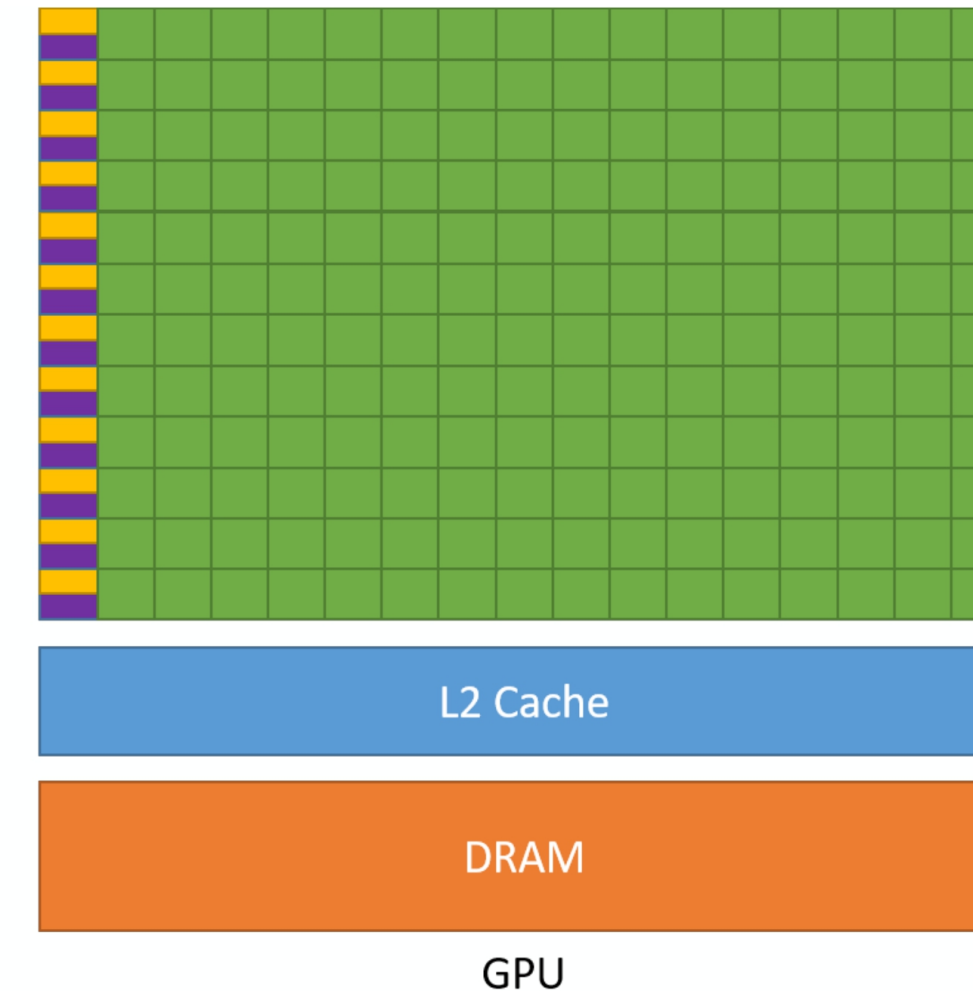
- DL exposes huge amounts of **similar, independent** numerical work:
 - Matrix multiplication: $A = CB$
 - Convolution: $K * A$
 - Attention: $\text{Softmax}(\mathbf{QK}^T)\mathbf{V}$
 - Vector operations\residuals, you name it



CPUs vs GPUs: different design goals



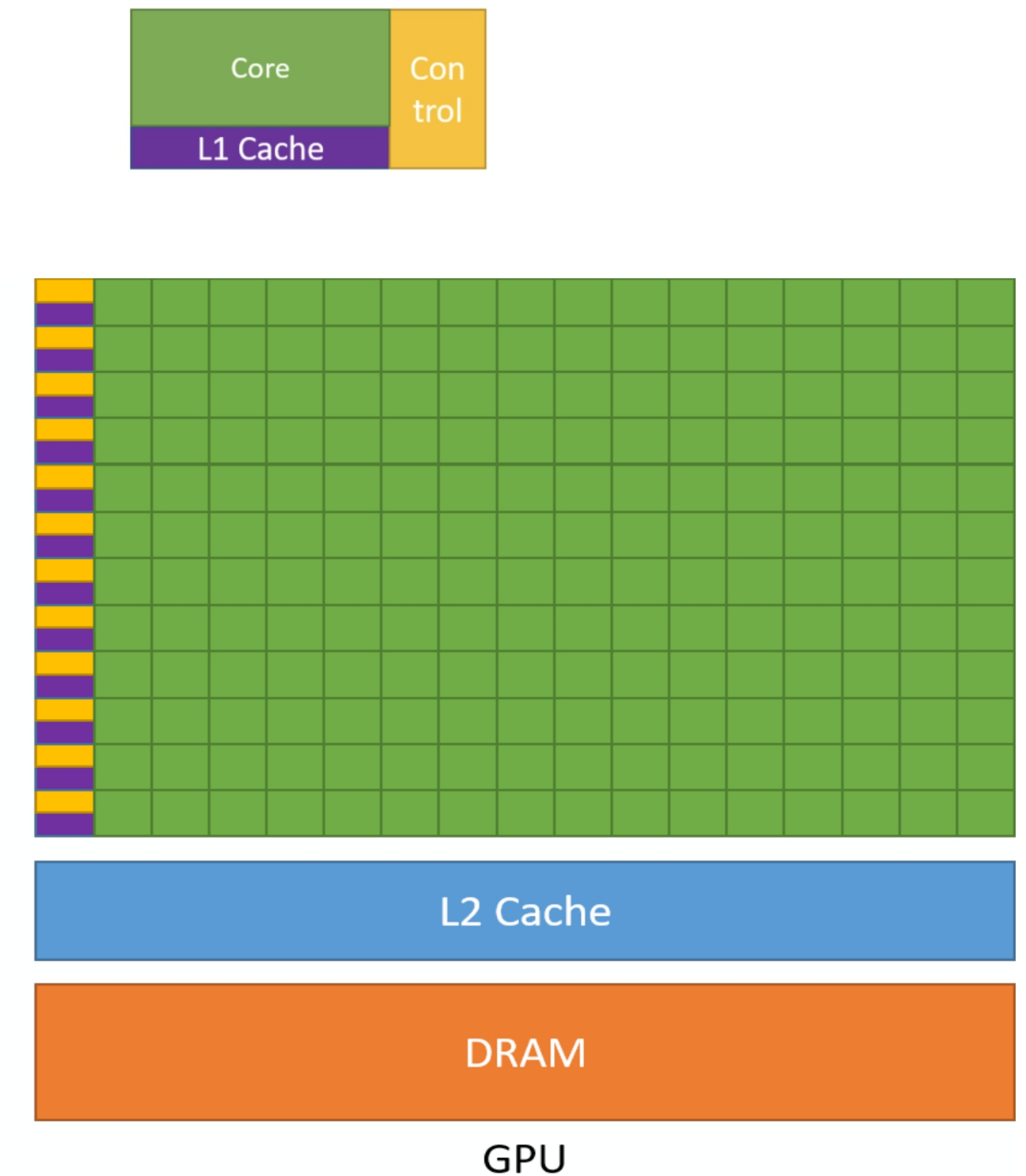
- Few powerful cores
- Large caches
- Sophisticated control (branch prediction/OoO execution)
- Excel at **irregular** or sequential work



- Many execution units, massive number of threads
- High Memory bandwidth, more transistors for data processing
- Shared control of [groups of] threads
- Excel at **simpler, regular** numerical work

CPUs vs GPUs: different design goals

- **Massive parallelism**
 - Many execution units, massive number of threads
 - Large amount of independent numerical work on thousands of CUDA cores
 - The control of each thread is much simpler → more energy-efficient
- **High Throughput**
 - Much portion of the chip is dedicated to arithmetic throughput and high-bandwidth data movement
- **Hardware specialization**
 - Modern GPUs add matrix engines and data movement hardware to specifically target DL workloads



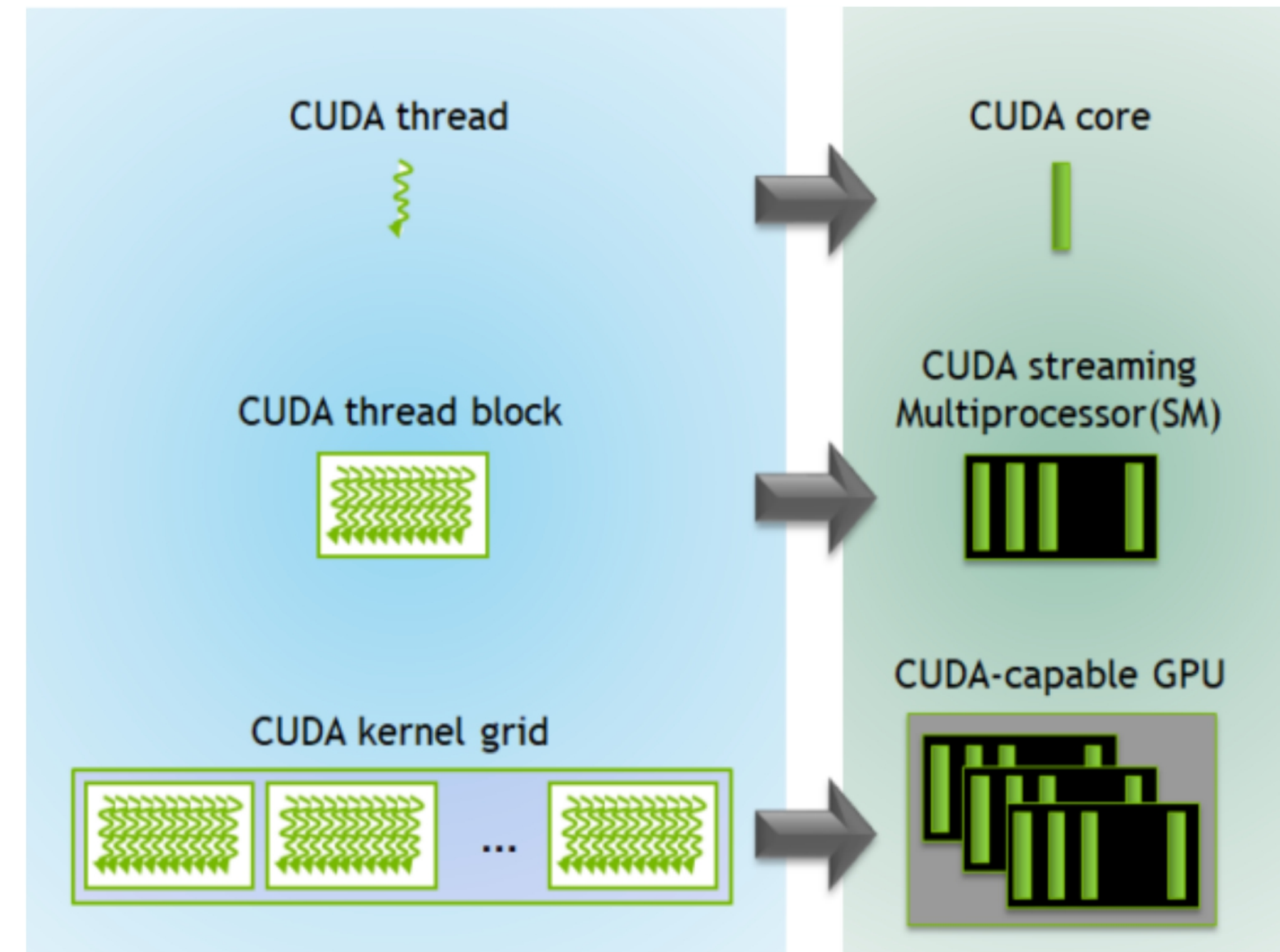
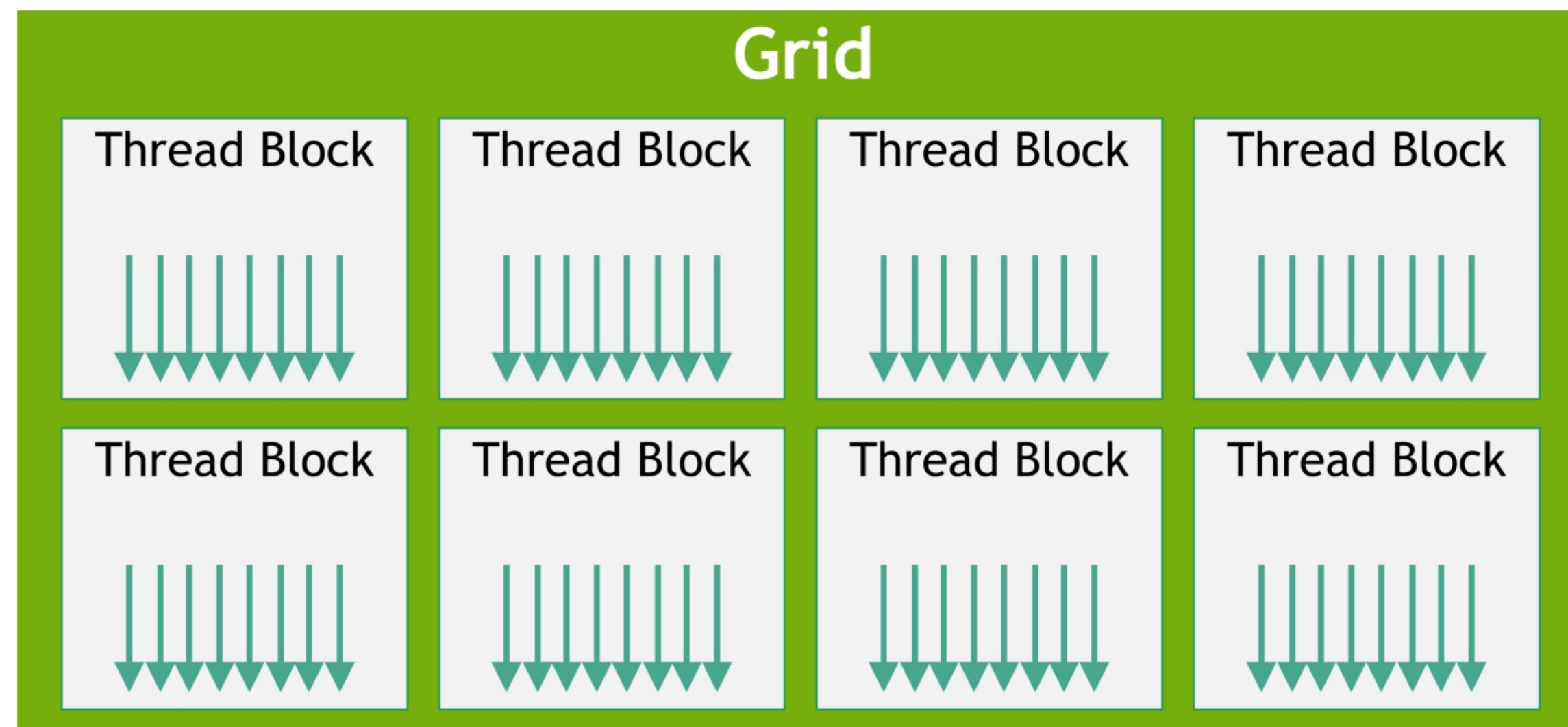
Streaming Multiprocessor (SM)

- Basic building unit of NVIDIA GPU
- Contains computational and data loading logic:
 - Arithmetical units
 - Registers
 - Warp schedulers (for issuing instructions)
 - Cache Units
 - Cooperation logic (shared memory)
 - Specialized units (Tensor Cores, Tensor Memory Accelerator)



Execution Hierarchy

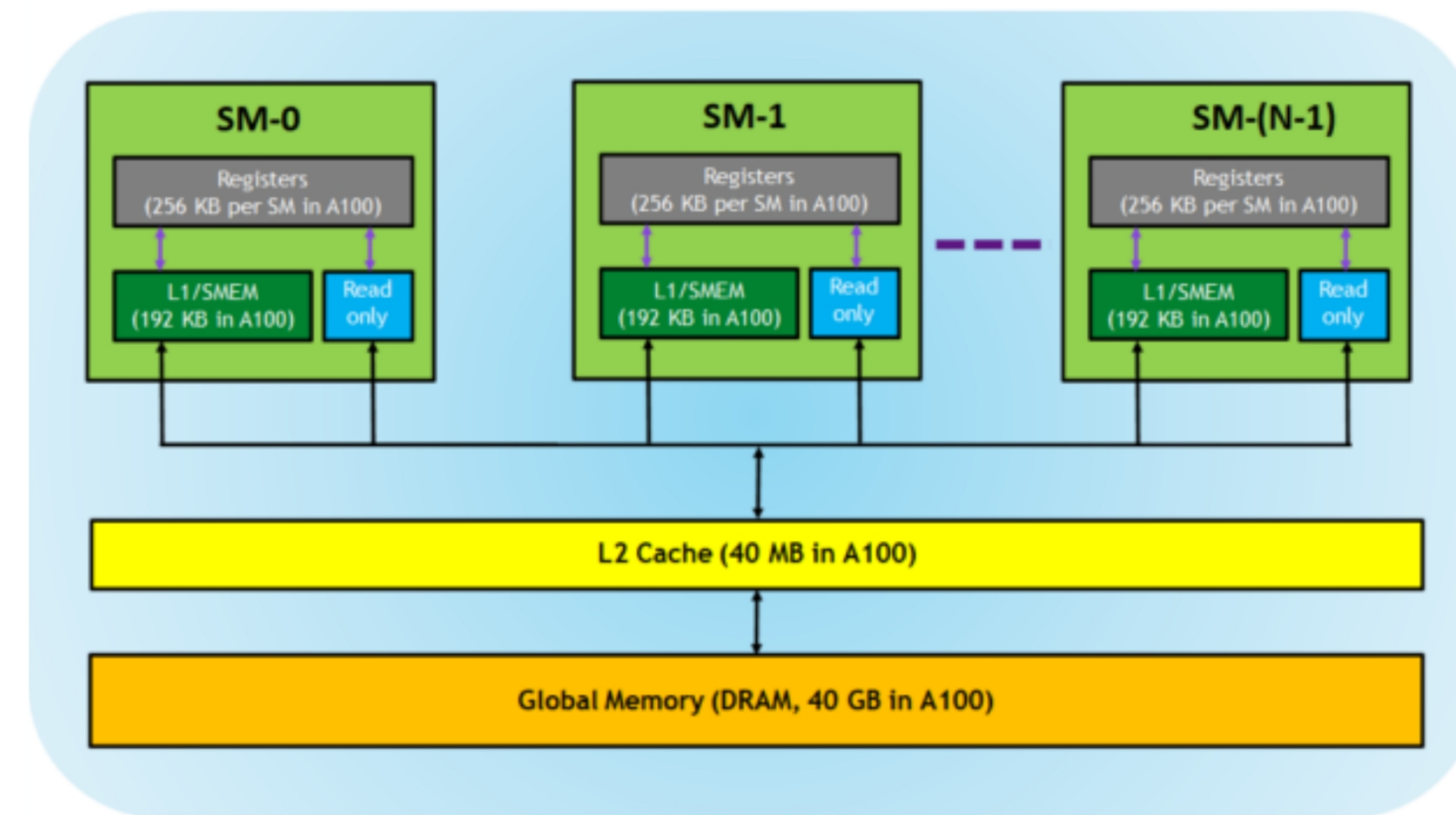
- Kernel is launched with multiple **threads**
- **Threads** are grouped into **thread blocks**
 - Each thread block is launched on a single SM
 - Number of thread blocks (threads) on SM is limited
 - Thread blocks are required to execute independently of one another
 - Threads within a thread block can cooperate
- Thread blocks form a **grid** for a kernel launch



GPU Memory Hierarchy

- GPU memory consists of several layers with varying scope, capacity and cost
- **Global** memory (HBM, high-bandwidth memory) - a lot, but **slow**. Frequently accessed elements are stored in L2 cache
- **Shared** memory - shared across SM - *much* smaller, but **faster**
- Runtime variables are stored in **registers** - **thread-specific storage**

Memory access can bottleneck execution



Memory Hierarchy of NVIDIA A100 40GB

Understanding GPU Performance

What makes GPU kernels fast?

- Thousands of threads are necessary - but not sufficient
- In terms of **parallelism**:
 - Do we expose enough independent work to keep the GPU busy?
- In terms of **data movement**:
 - Can we feed execution units without wasting bandwidth or waiting IO?
- In terms of **specialization**:
 - Are we using the hardware path designed for this specific operation?

Latency and Throughput

Latency

- How long does operation takes

Request → Result

- A single request may still take thousands of cycles

Throughput

- How much work completes per unit time (capacity)

e.g. Request per second

- GPUs are designed to maximize sustained throughput

GPU performance is primarily a throughput problem

Latency Hiding

- When having multiple warps, we can hide the huge latency of IO operations by switching between them
- When one warp waits, warp scheduler can run another ready warp
- Switching between warp is near free, so warp not waiting for the IO can compute while others can wait



We need many resident warps

- GPUs tolerate long-latency operations by keeping many independent warps in flight (**latency hiding**)

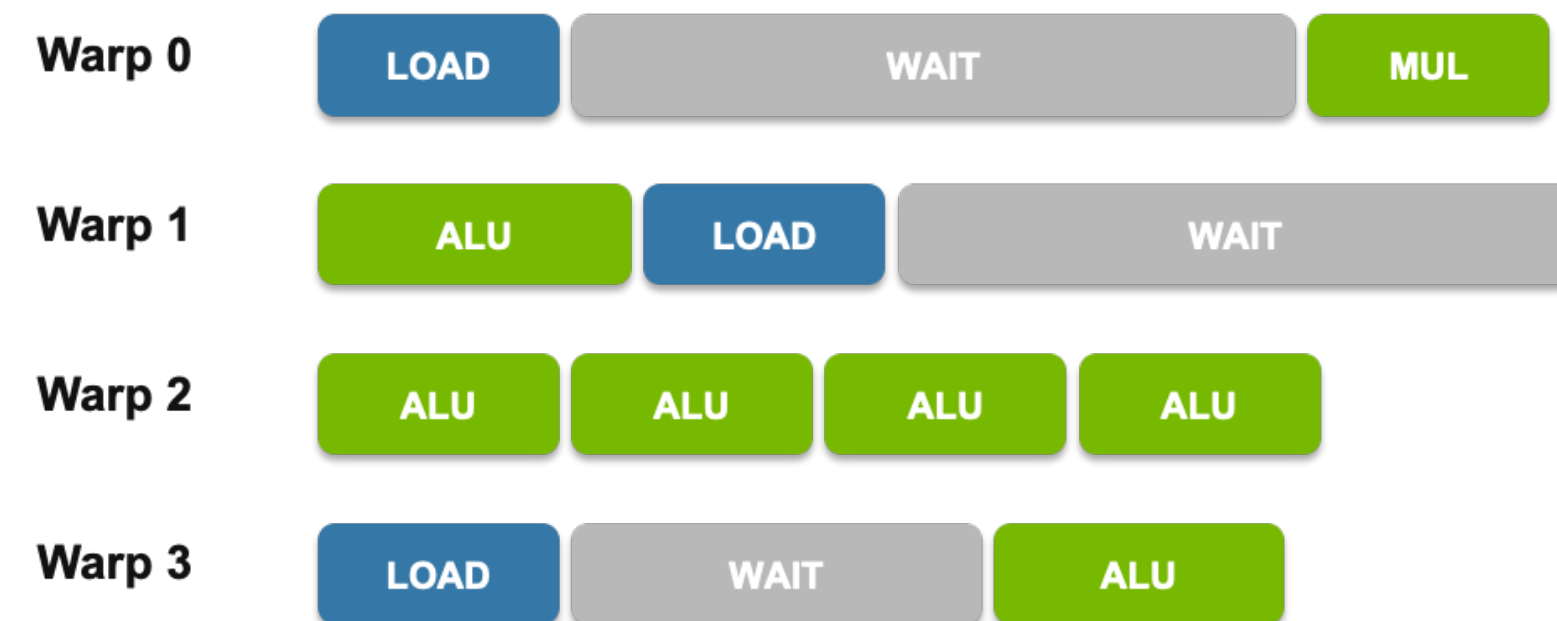
- One warp:

```
x = A[i]; // load
y = x * 2; // depends on x
```



The dependent instruction cannot execute until its operand is ready

- Warp scheduling hides latency by switching between stalled and ready warps:



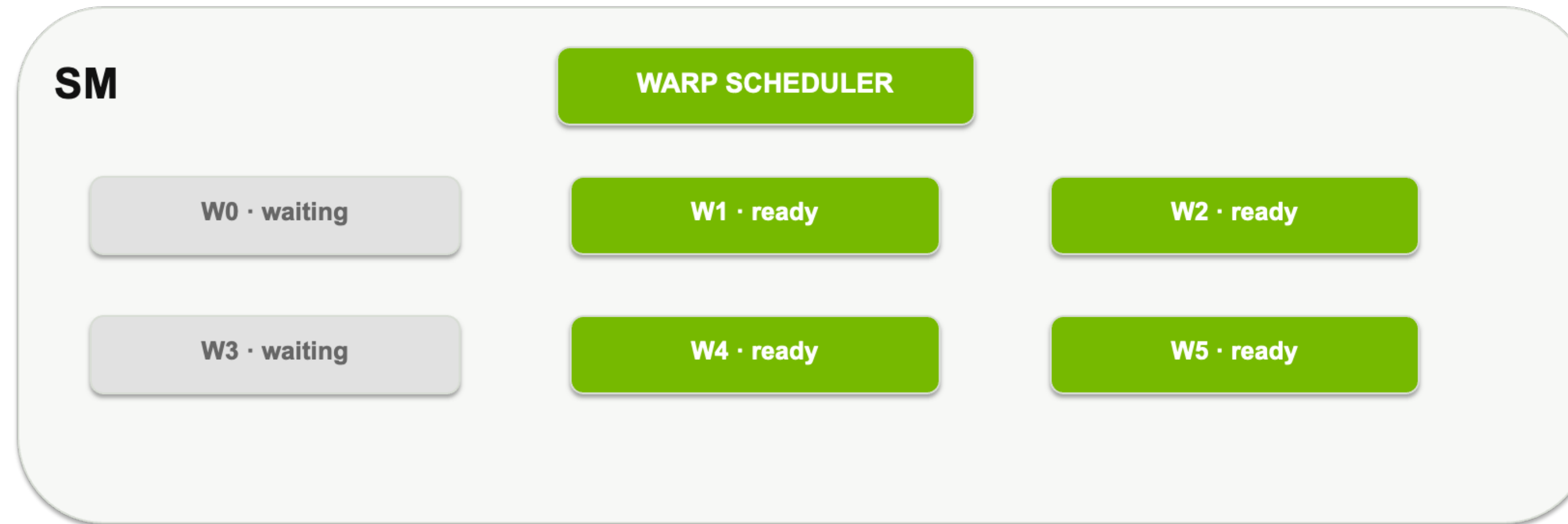
Warp scheduler

At each issue opportunity:

1. Find a resident warp
2. Whose next instruction is ready
3. Issue it

Latency hiding = do useful work while another warp waits

We need many resident warps



More independent resident warps → more chances to find ready work

Occupancy

- Occupancy:
$$\frac{\text{\# of resident warps}}{\text{max. \# of supported active warps}}$$
- Ratio of resident warps on the SM
- Residency metric, not a direct performance metric
- Example:
 - Max supported 64 warps
 - Resident now: 32
 - Occupancy is 50%

What limits occupancy?

An SM has finite resources that resident blocks need

Registers per thread

More registers per thread → fewer warps may fit

Shared memory per block

Larger SMEM tiles → fewer blocks may fit

Threads per block

Block size interacts with warp and block limits

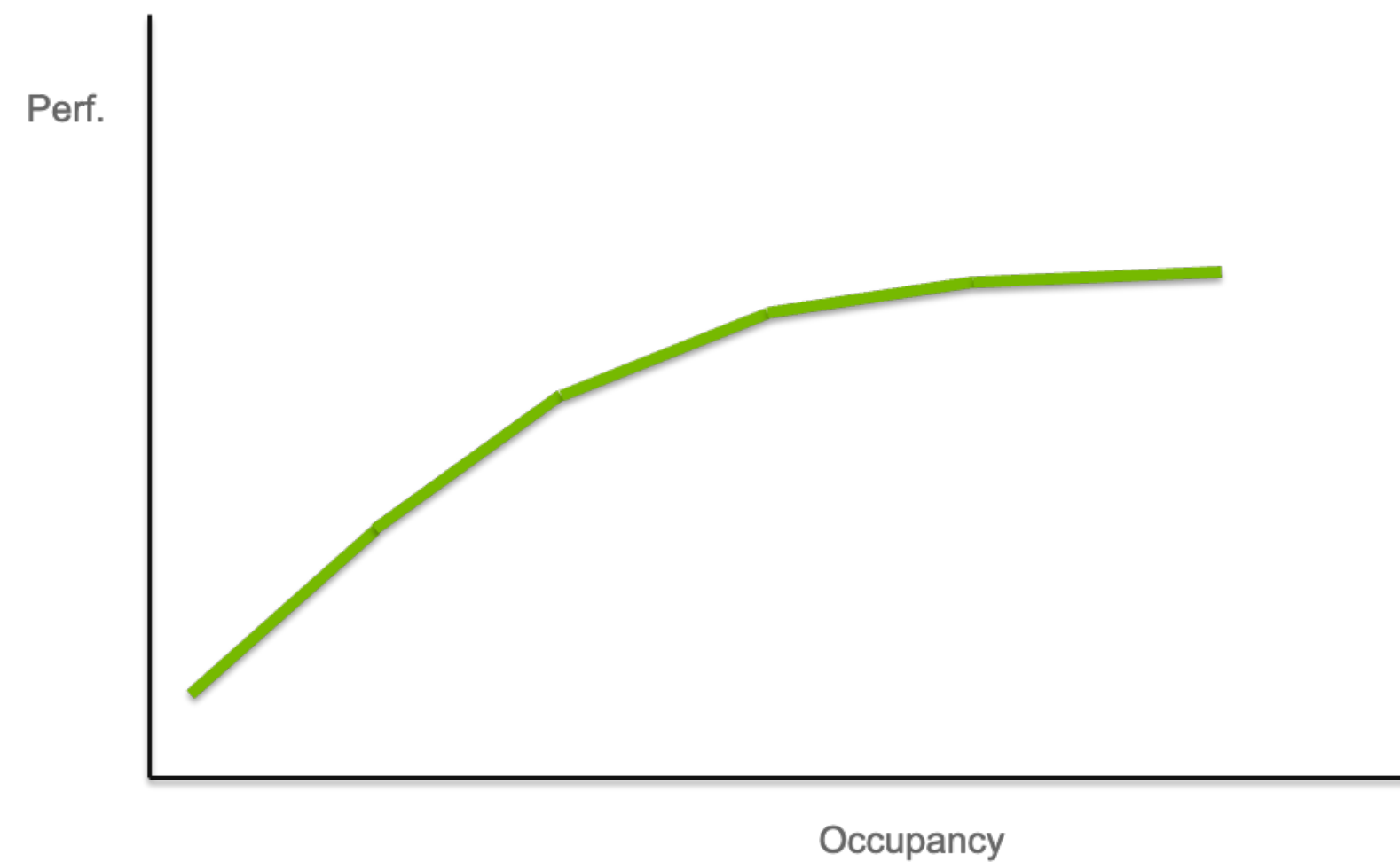
Architectural limits

Maximum resident warps, blocks per threads are finite

Resource usage per block increases $\implies$ possible residency decreases

Max occupancy $\neq$ max performance

- Occupancy is a means to hide latency, latency $\neq$ performance
- Initially - more warps can hide more memory
- Eventually - execution pipelines are already fed, extra residency may not help



Max occupancy $\neq$ max performance

- Use just enough occupancy to hide latency
- Spend remaining resources to make each warp efficient

Where is the bottleneck?

Two fundamental hardware limits

Once enough work is in flight, performance is bounded by the rate at which hardware can move data or execute math

Compute throughput

How many arithmetic operations can the GPU execute per second?

Examples: FP32 FLOP/s, Tensor Core FLOP/s

Memory bandwidth

How many bytes can the memory system deliver per second?

Examples: HBM Bandwidth, L2 Bandwidth

Does your kernel spend its time computing or moving data?

Example: vector addition

$$C[i] = A[i] + B[i]$$

Compute throughput

1 floating-point addition $\approx$ 1 FLOP per element

Memory bandwidth

Read A & B, Write C: $(4 + 4 + 4)B = 12B$

$$\approx \frac{1}{12} = 0.083 \text{ FLOP per byte moved}$$

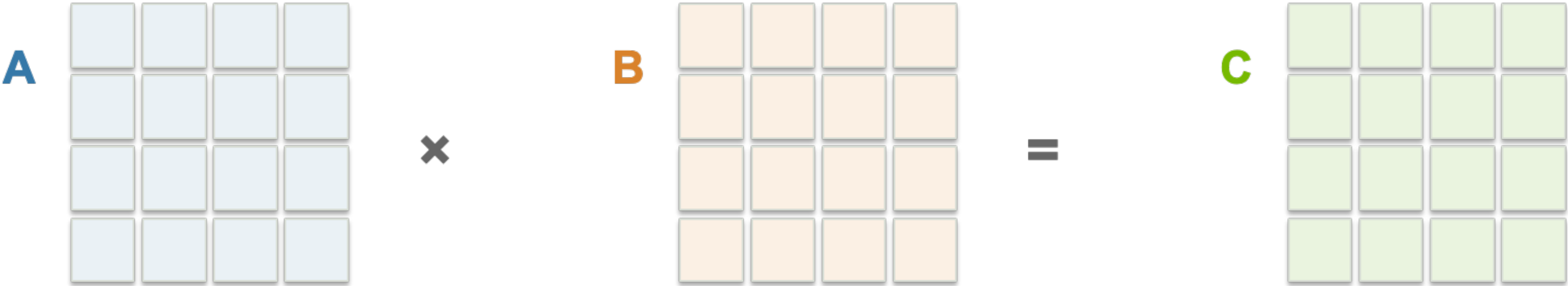
Doubling compute throughput alone won't help

Example: tiled matrix multiplication

$$C = AB$$

Tile size T ; The same loaded values can participate in many multiply-accumulate operations

Thread block of size $T \times T$: each thread load one element of A and one element of B into SMEM, then accumulates results in SMEM



Compute throughput

$$2T \cdot \left\lceil \frac{N}{T} \right\rceil - T \text{ sums, } T \text{ additions inside each tile}$$

Memory bandwidth

$$8 \cdot \left\lceil \frac{N}{T} \right\rceil + 4 - \text{each thread reads 4B-elements of A and B for each tile, then writes one 4B-result}$$

$$AI = \frac{2T \cdot \left\lceil \frac{N}{T} \right\rceil}{8 \cdot \left\lceil \frac{N}{T} \right\rceil + 4} = \frac{T}{4 \cdot \left(1 + 2 / \left\lceil \frac{N}{T} \right\rceil \right)} \text{ FLOP per byte moved;}$$

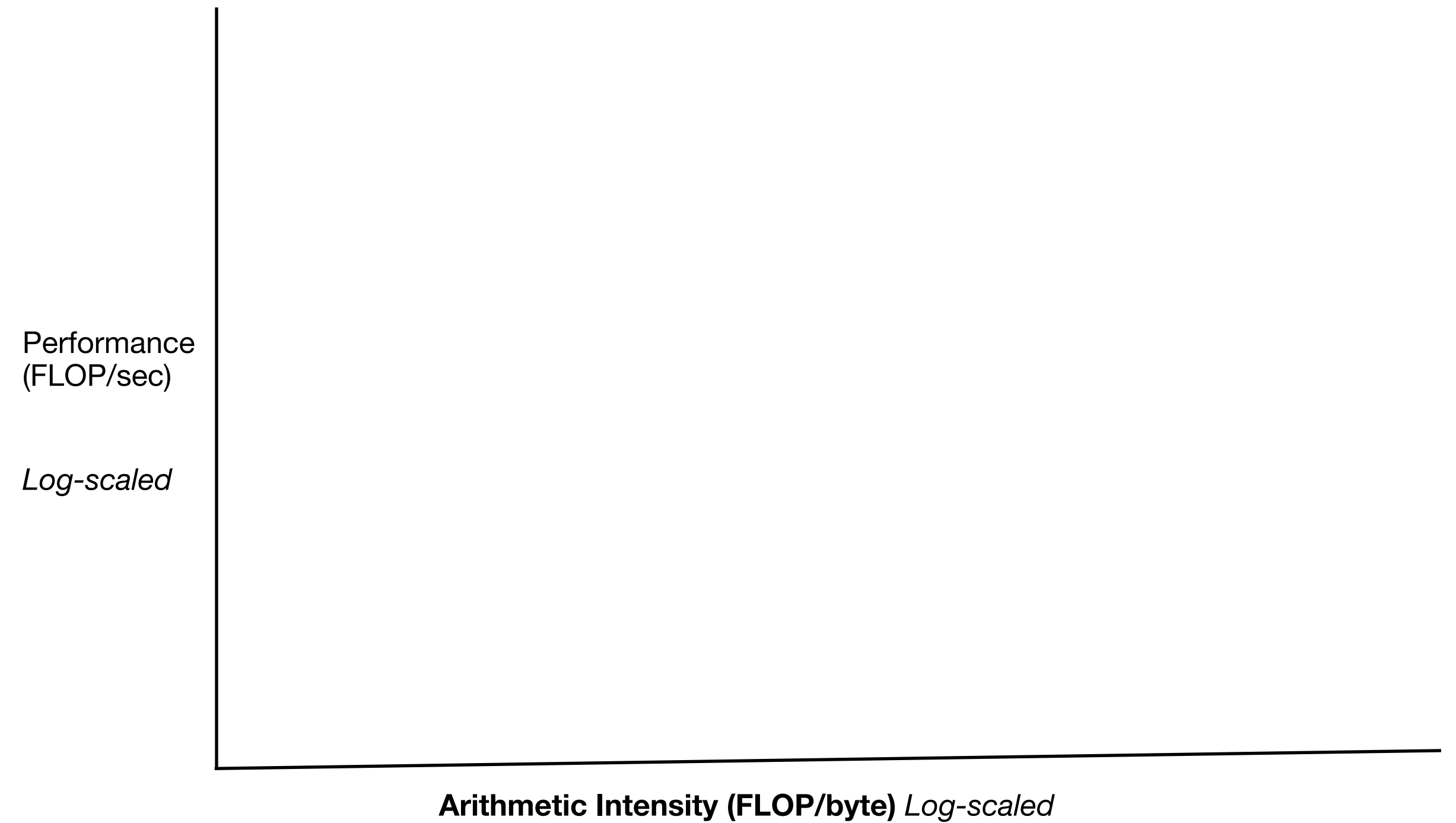
Increasing tile size increases AI

The roofline model

- Arithmetic intensity provides isolated numbers
- But how do we know how fast we are?
- The roofline model can provide an answer

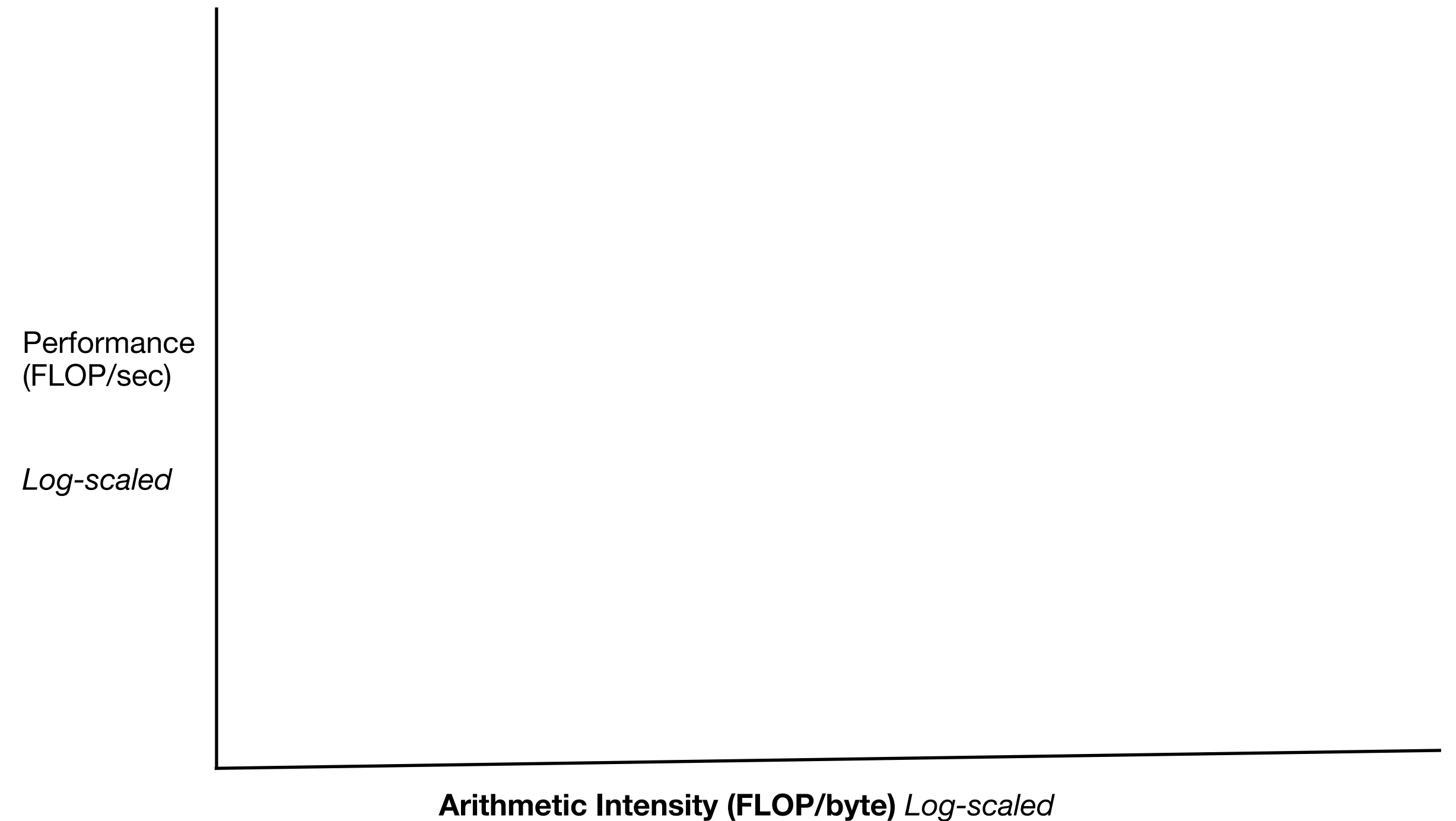
The roofline model

- Arithmetic intensity $AI = \frac{\text{FLOP}}{\text{Byte}}$



The roofline model

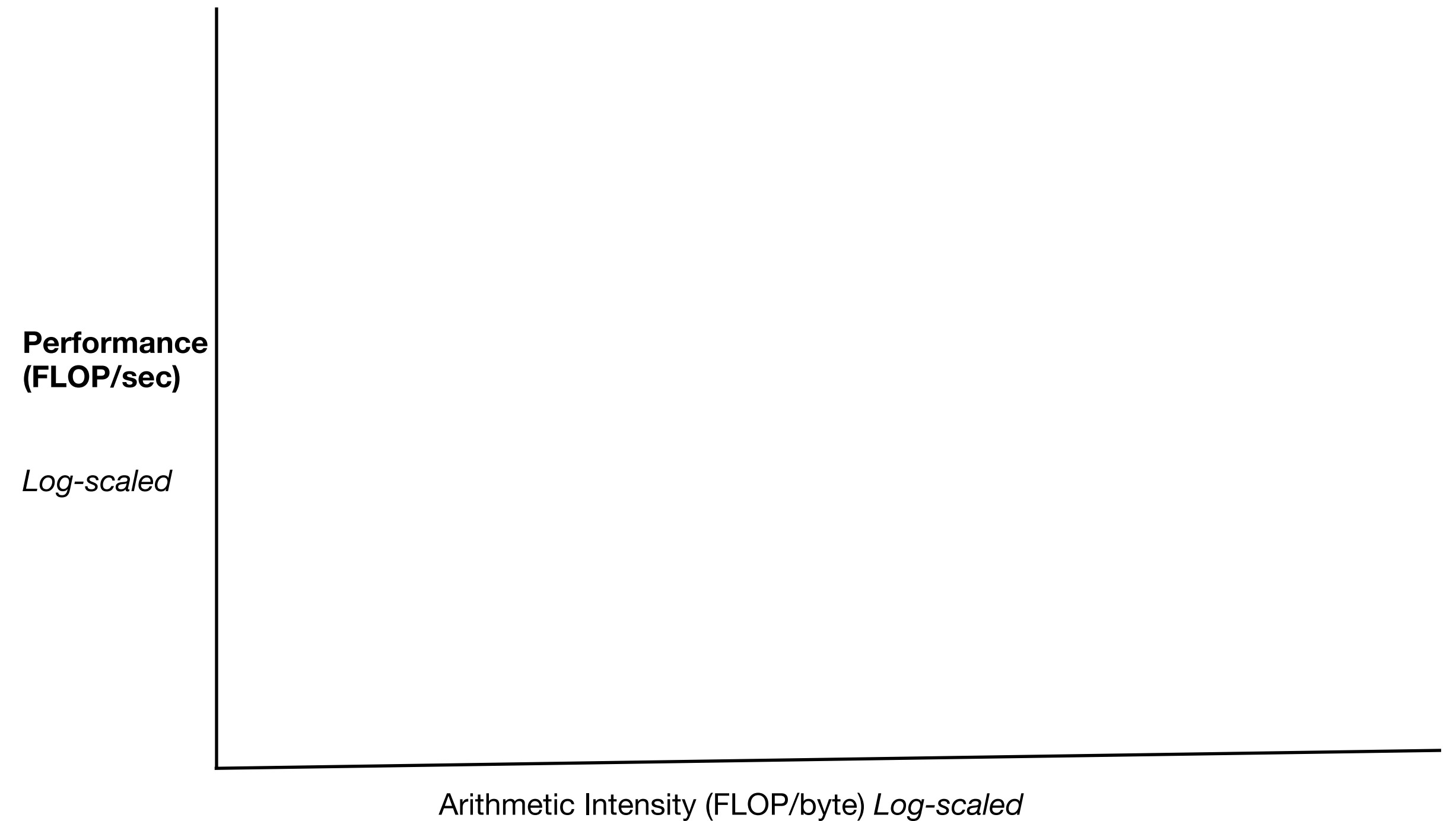
- Arithmetic intensity $AI = \frac{\text{FLOP}}{\text{Byte}}$
- Max memory bandwidth
 $BW = \frac{\text{Byte}}{\text{sec}}$



The roofline model

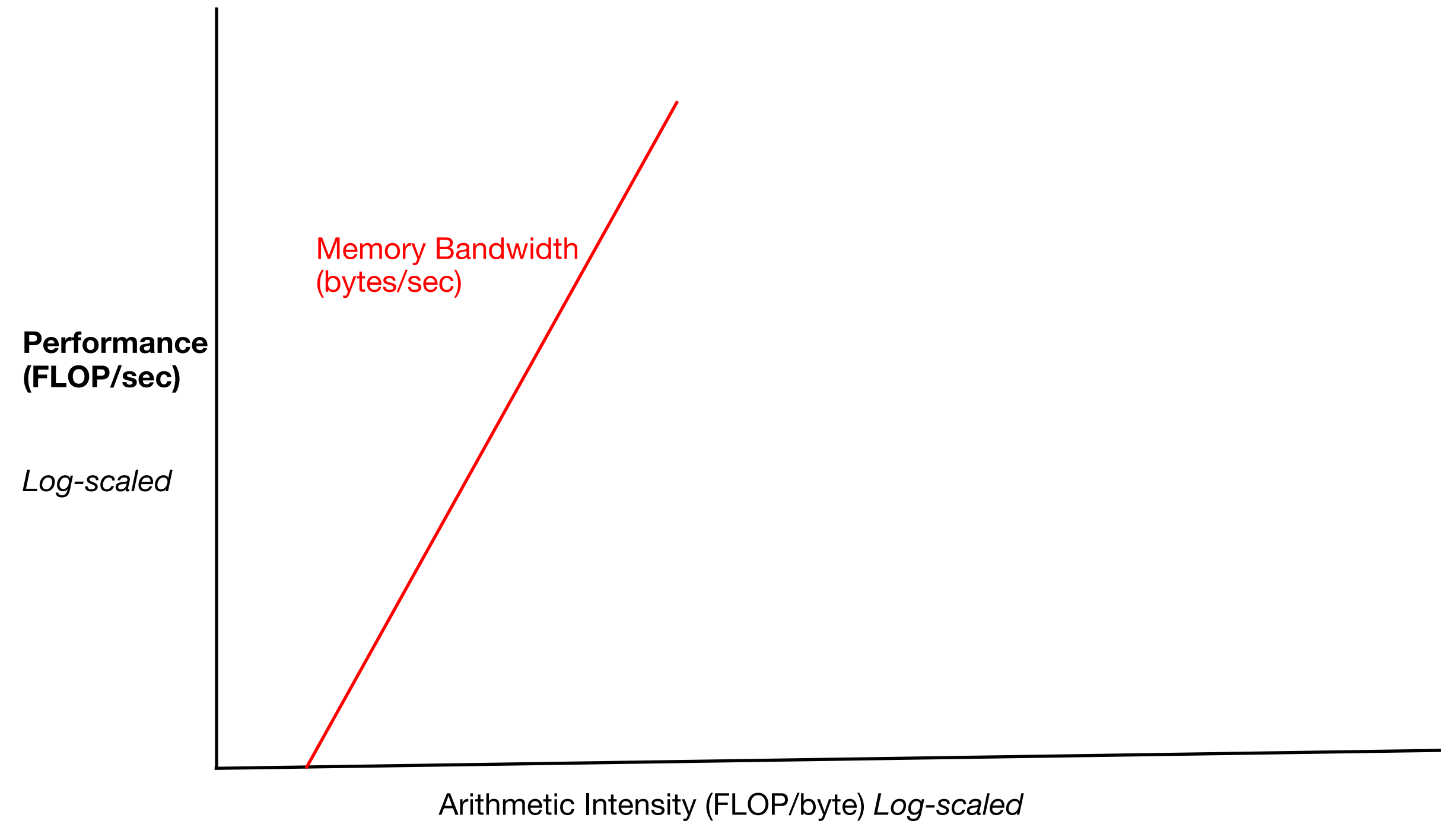
- Let's compute our performance:

$$P = AI \cdot BW = \frac{\text{FLOP}}{\text{Byte}} \cdot \frac{\text{Byte}}{\text{sec}} = \frac{\text{FLOP}}{\text{sec}}$$



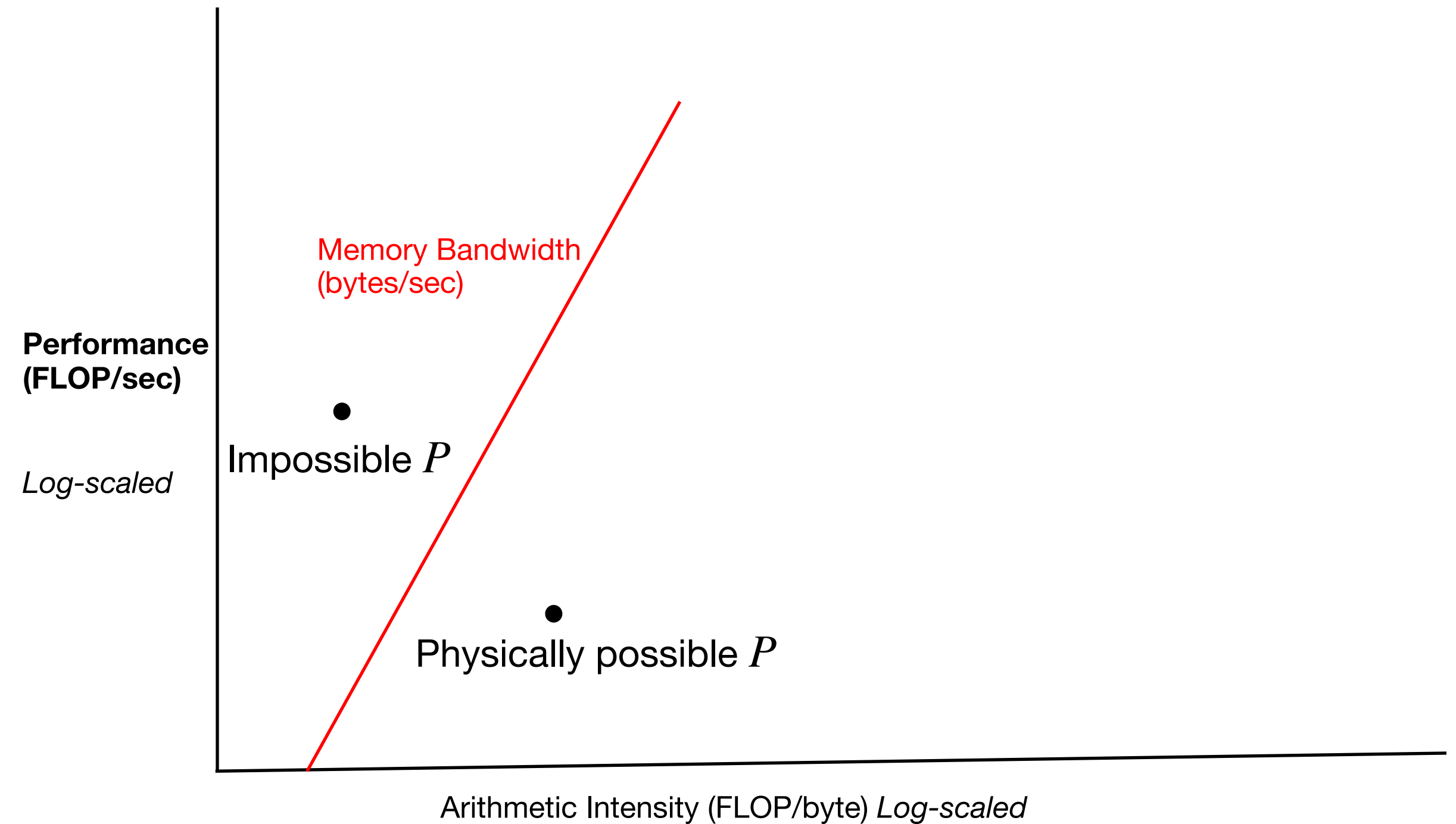
The roofline model

- BW has physical limit
 - At some point we saturate memory bandwidth



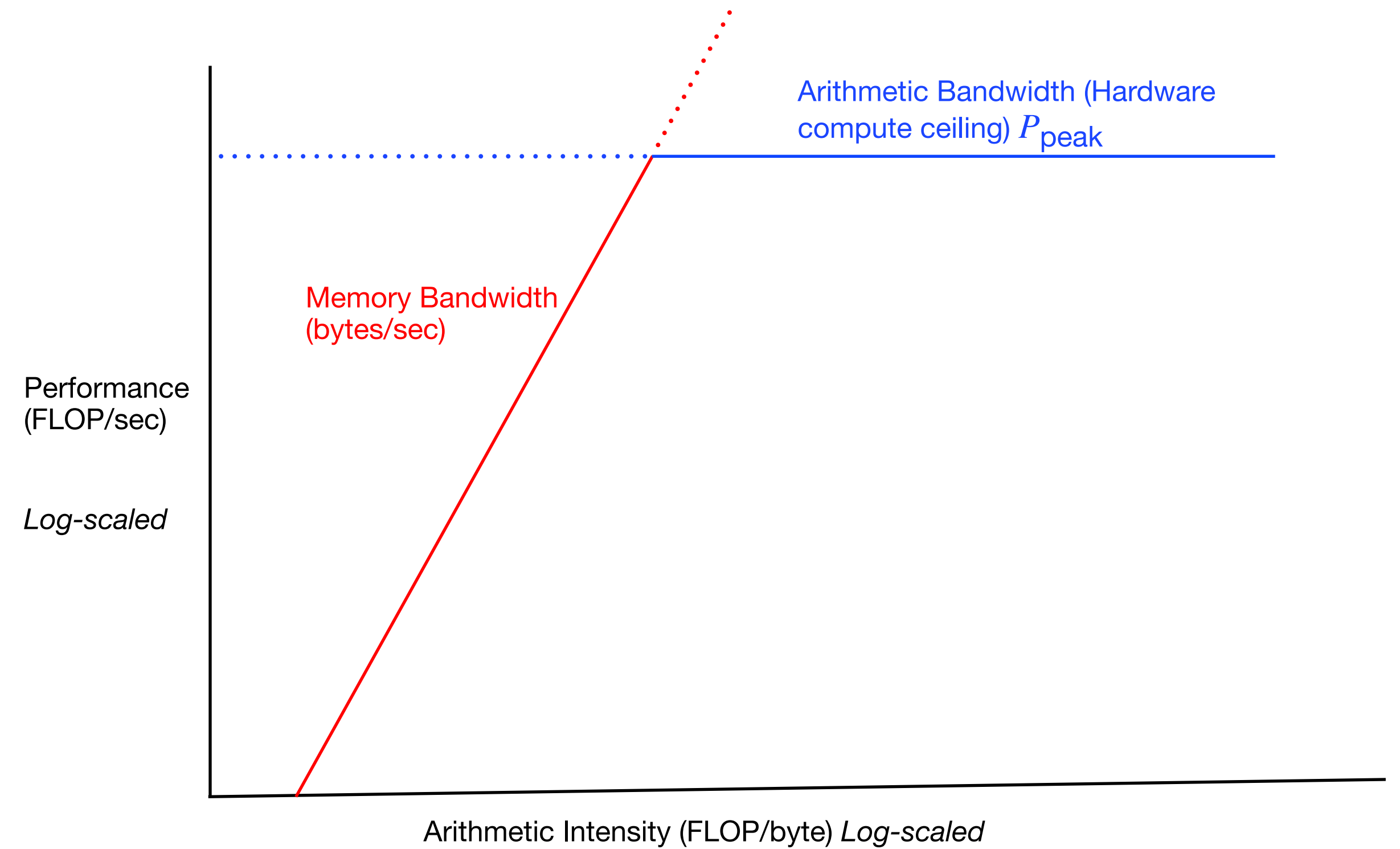
The roofline model

- BW has physical limit
- At some point we saturate memory bandwidth
- BW limits P for each AI



The roofline model

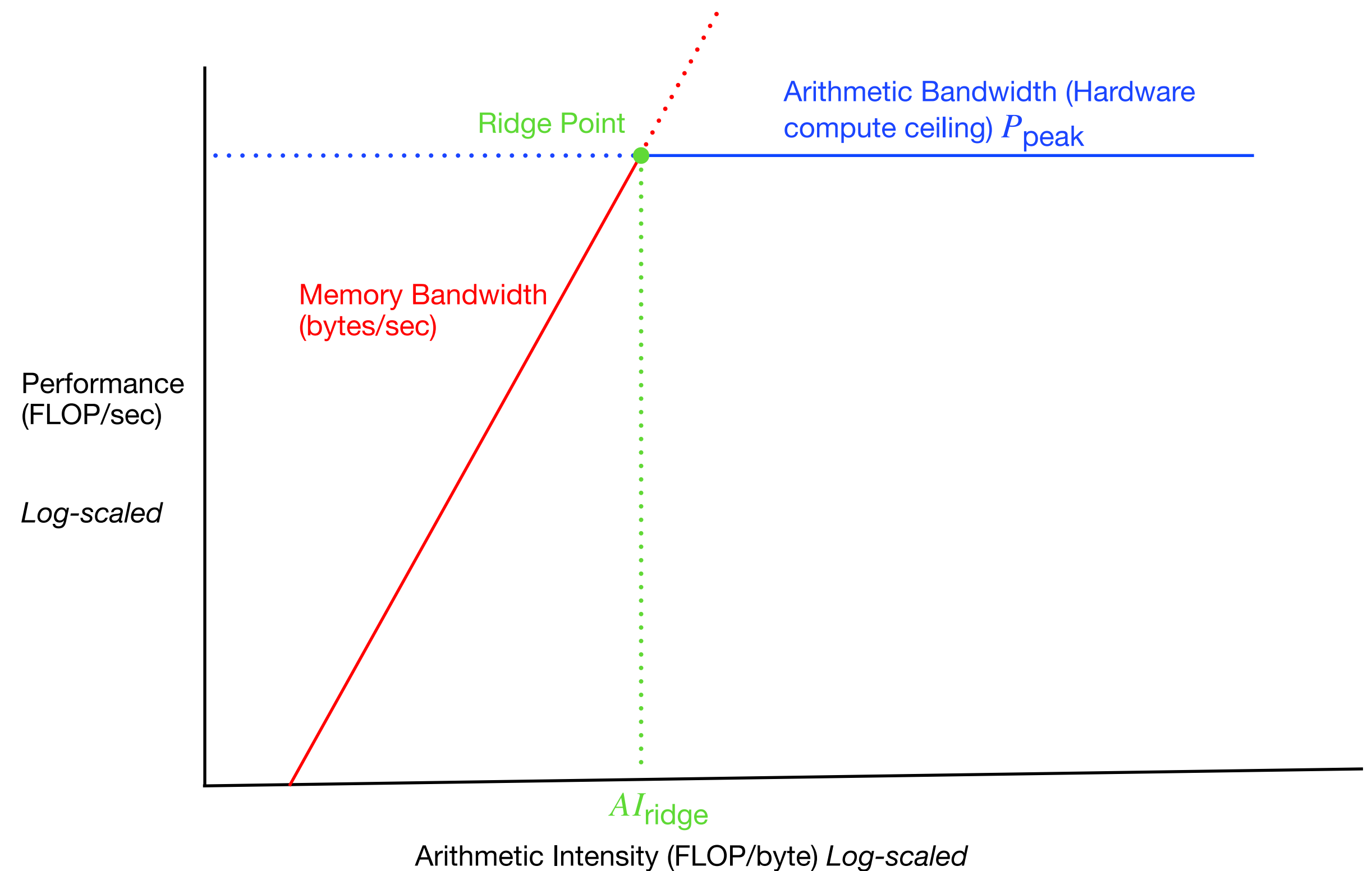
- GPU has P_{peak} - max achievable performance



The roofline model

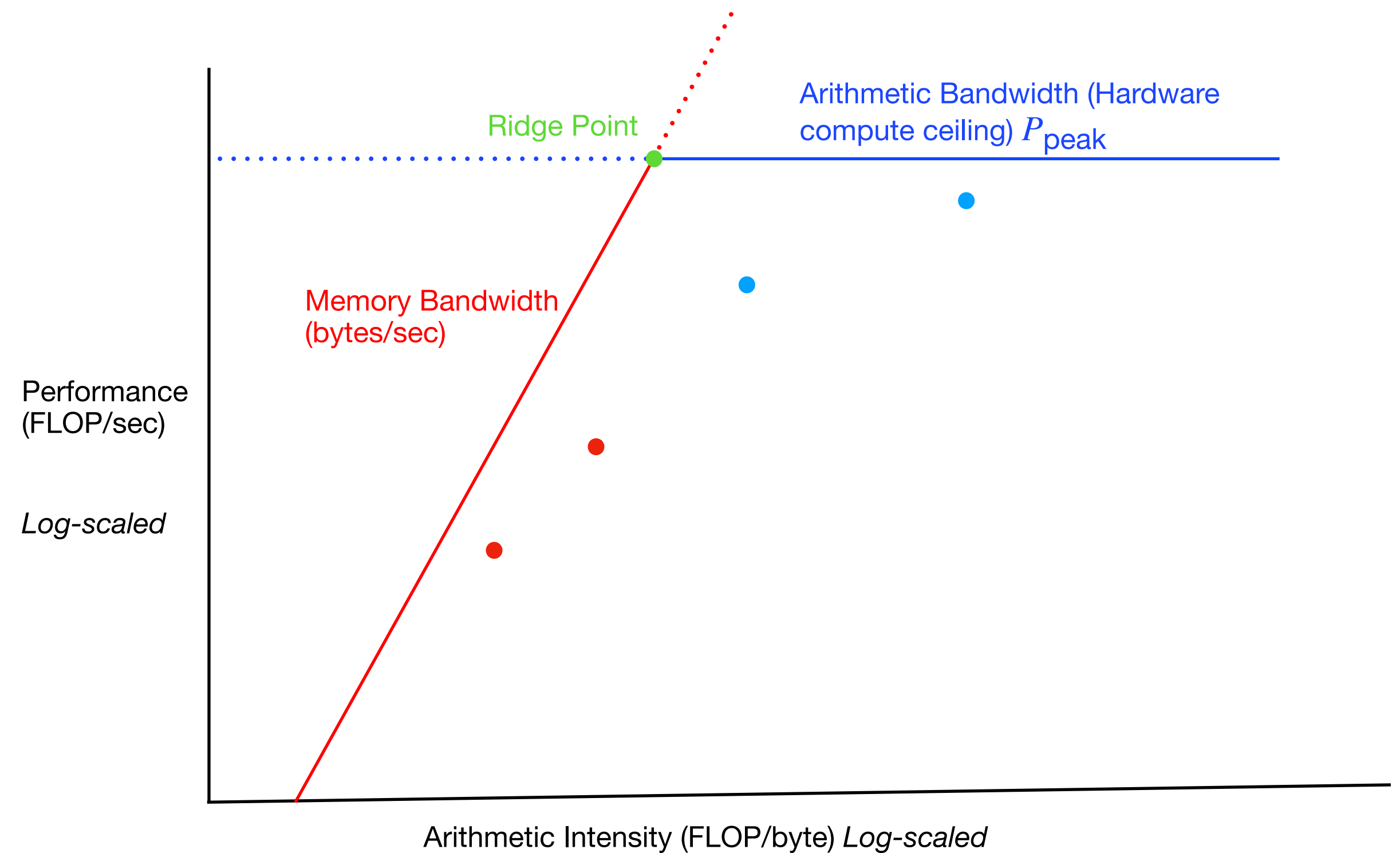
- GPU has P_{peak} - max achievable performance
- **Ridge point** - Arithmetic intensity threshold operation:
 - Stops being bounded by memory bandwidth
 - Becomes bounded by hardware performance

- $$AI_{\text{ridge}} = \frac{P_{\text{peak}}}{BW}$$



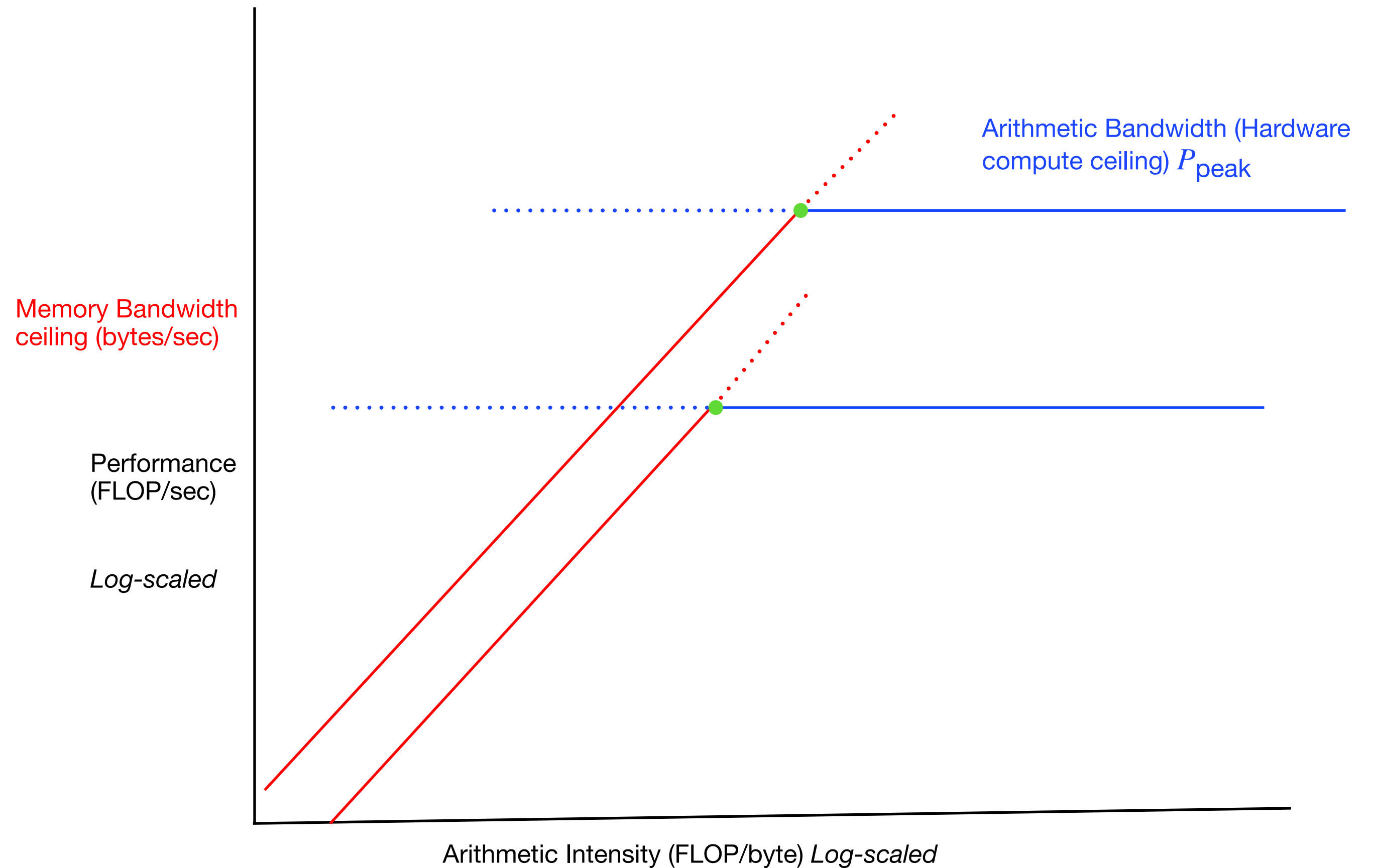
The roofline model

- **Ridge point** - Arithmetic intensity threshold where workloads become compute-bound
- **Memory**-bound
- **Compute**-bound



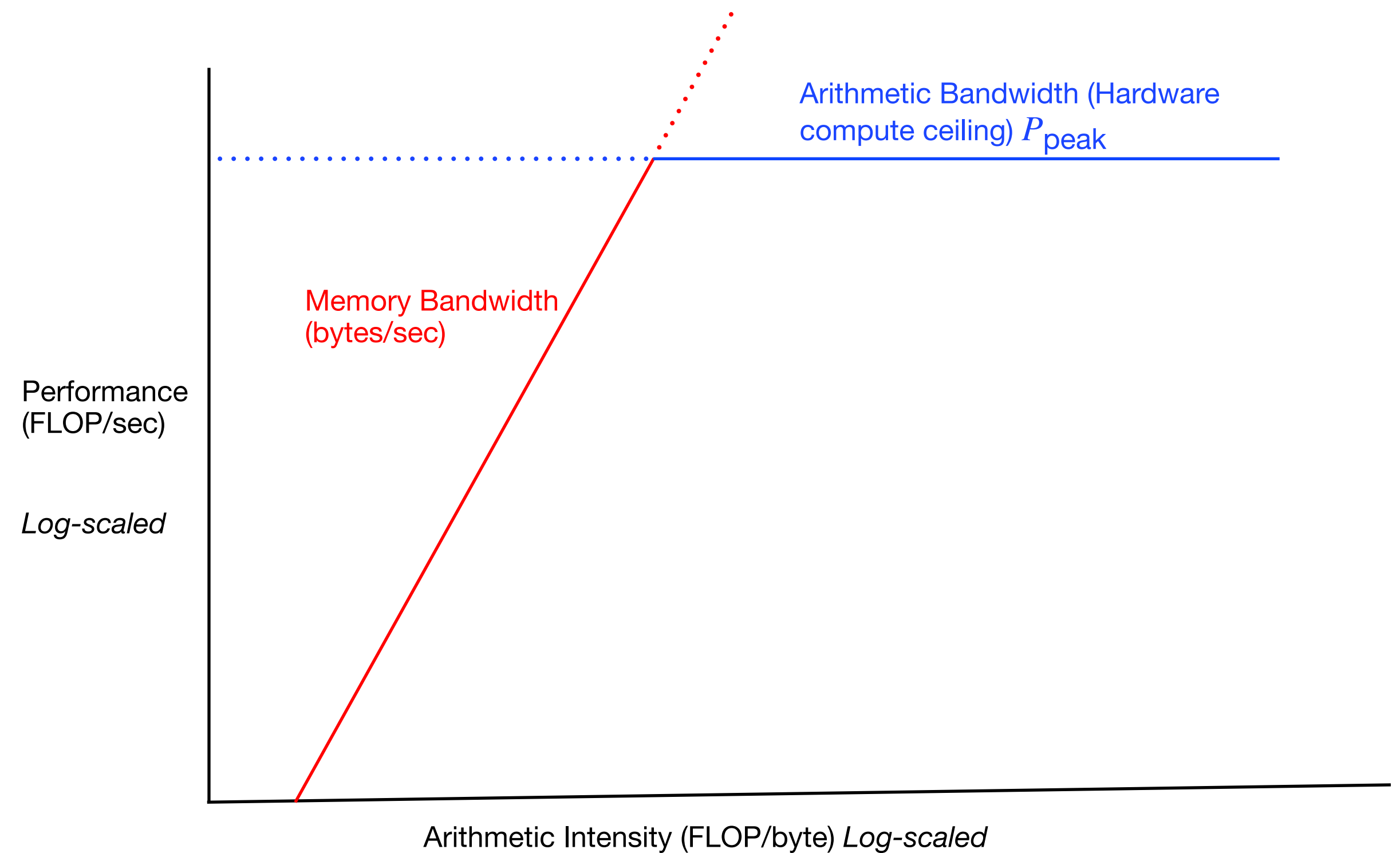
The roofline model

- **Ridge point** - Arithmetic intensity threshold where workloads become compute-bound
- **Memory**-bound
- **Compute**-bound
- For different hardware ridge point & graph are different



The roofline model

- Connects kernel performance with hardware characteristics
- Helps to determine the bottlenecks



Improving efficiency in Deep Learning

Where the overhead comes from

- GPU underutilization
- Input shapes
- Kernel launch latency
- Host-device synchronization
- Unfused operations
- Device-to-Device or Host-to-Device data transfer

Where the overhead comes from

- GPU underutilization
 - Input shapes
 - Kernel launch latency
 - Host-device synchronization
 - Unfused operations
 - Device-to-Device or Host-to-Device data transfer
 - Global memory access, branch divergence, register occupancy, shared memory bank conflicts, atomic operations, cache misses, interconnect bandwidth
- Can be fixed from Python (or already handled by PyTorch)
 - Can be partially fixed from Python
 - Can't be fixed from Python. Can occur in custom code. Won't be covered

GPU underutilization

- Current batch size can underutilize SMs
- Too small thread blocks size or too large grid size can lead to smaller occupancy

GPU underutilization - Tensor cores

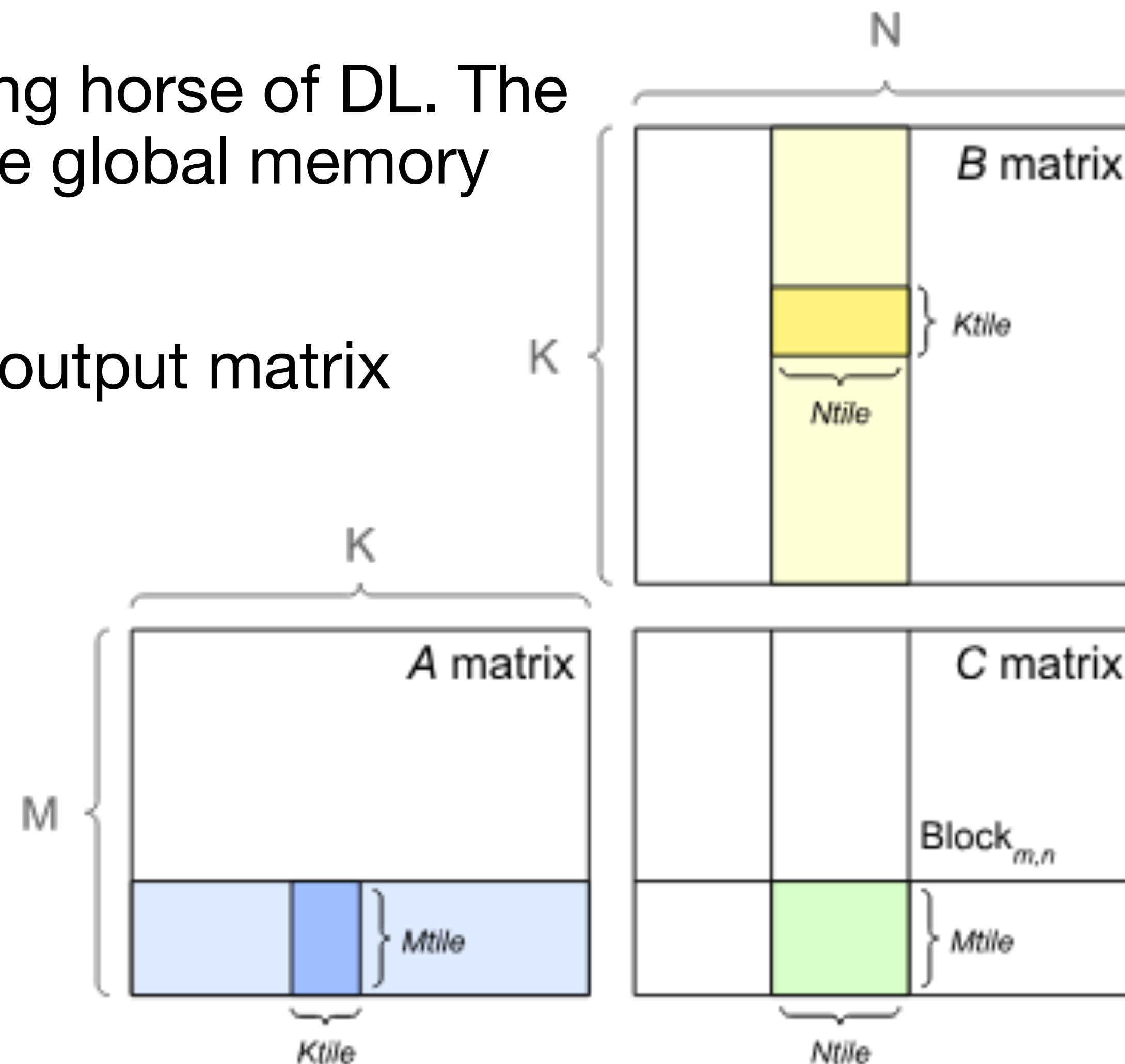
- GPUs have specific hardware for matrix multiplication - Tensor cores
- Provide much higher compute throughput for Matrix-Multiply-Accumulate operations: $D_{M \times N} = A_{M \times K} B_{K \times N} + C_{M \times N}$
- PyTorch already uses Tensor Cores internally

GPU underutilization - Mixed precision

- Modern hardware has much higher throughput for lower-precision (BF16/FP8/FP4)
- Using mixed precision increases utilization
- Quantization:
 - Reduces memory traffic
 - Increases throughput in low precision

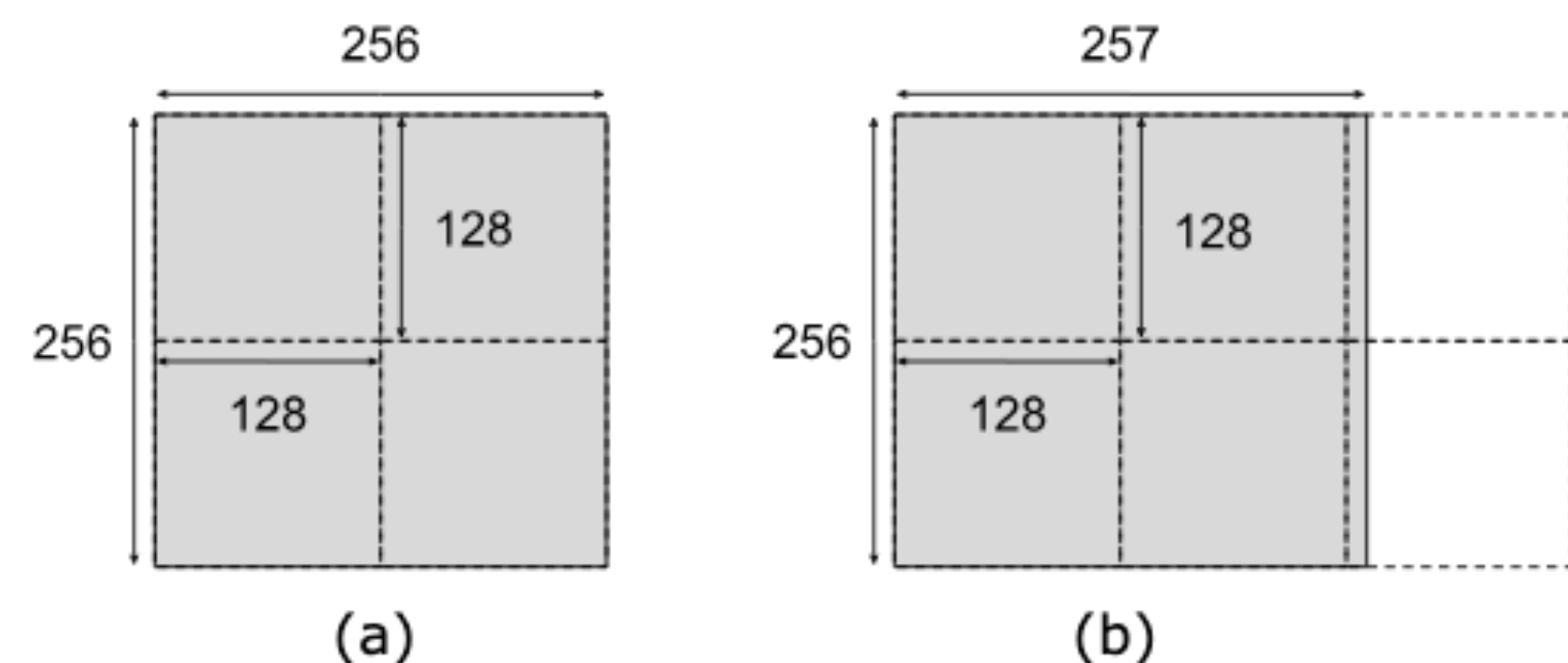
Inconsistent input shapes - Matmul example

- Matrix multiplication (*matmul*) is the working horse of DL. The algorithms for CUDA utilize **tiling** to reduce global memory accesses
- Each thread block computes its **tile** of an output matrix

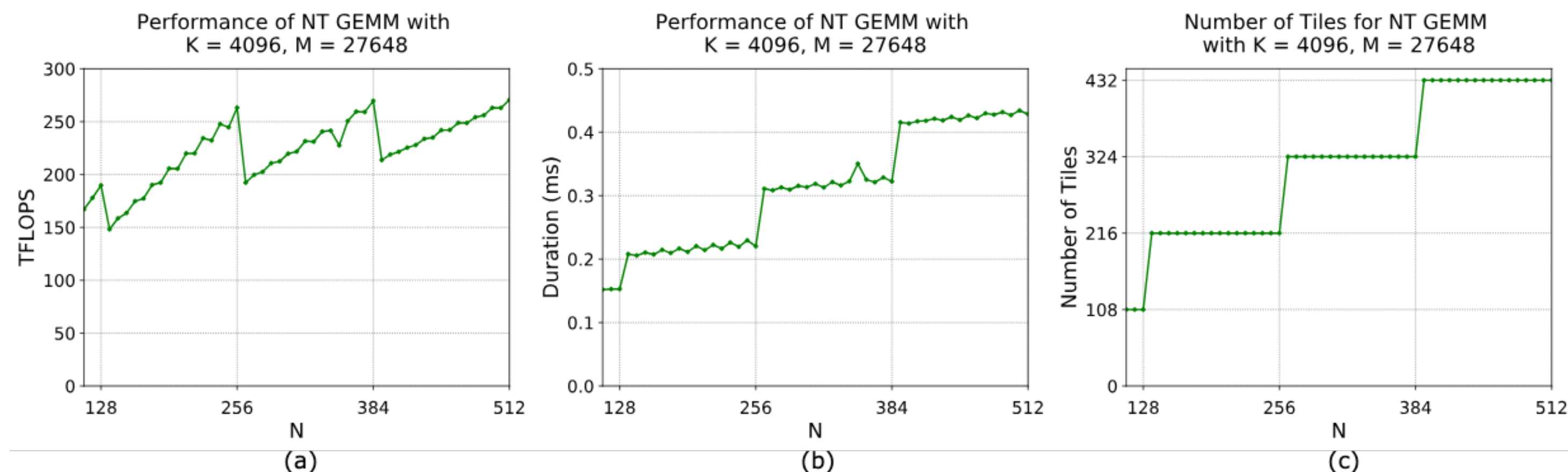


Inconsistent input shapes - Matmul example

- **Tile quantization** — matrix dimensions are not divisible by tile size
- Some thread blocks will do very little work on the corner cases



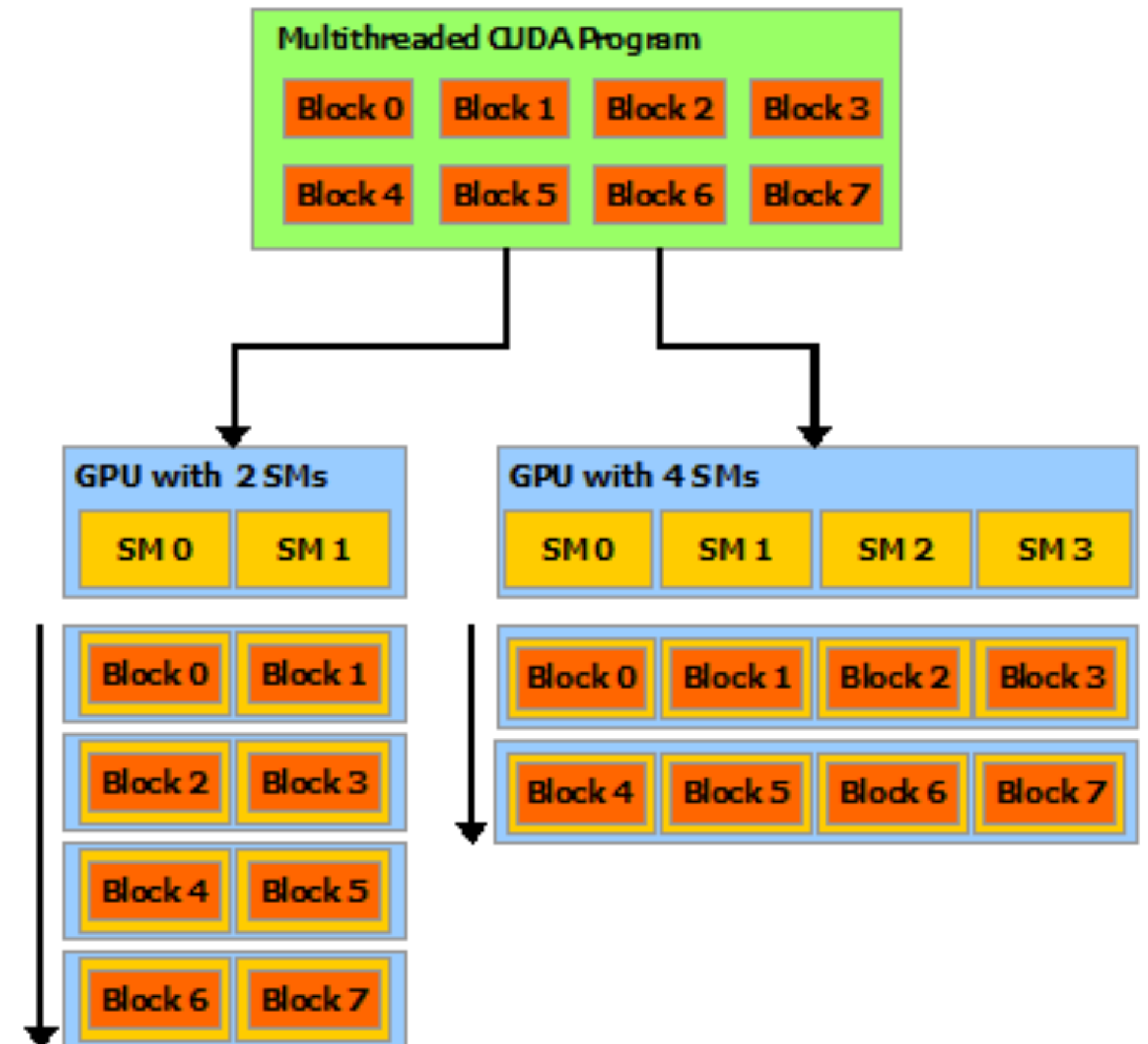
(a) Best case (b) 2 thread blocks will waste most of the work



Tile quantization effect in terms of (a) achieved FLOPS throughput and (b) elapsed time, as well as (c) the number of tiles created. Measured with a function that uses 256x128 tiles over the MxN output matrix.

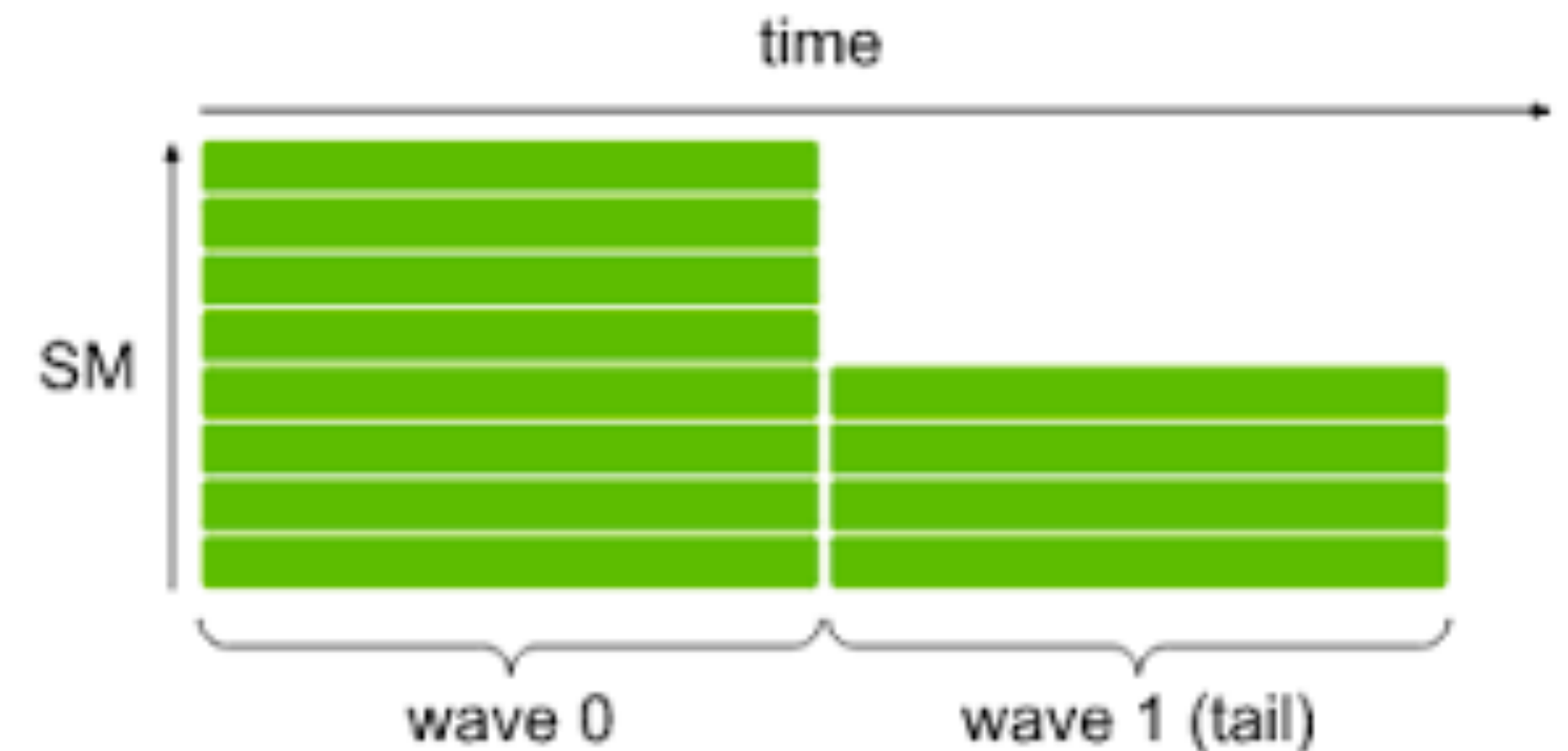
Inconsistent input shapes

- **Wave quantization** — total number of tiles (thread blocks) is quantized to the number of SMs.
- Kernel will be executed in waves.
- Number of waves is $\left\lceil \frac{\text{Tiles}}{\text{Num. of SMs}} \right\rceil$
- The last wave can underutilize the GPU
- Best utilization occurs with the number of tiles is the multiple of SMs



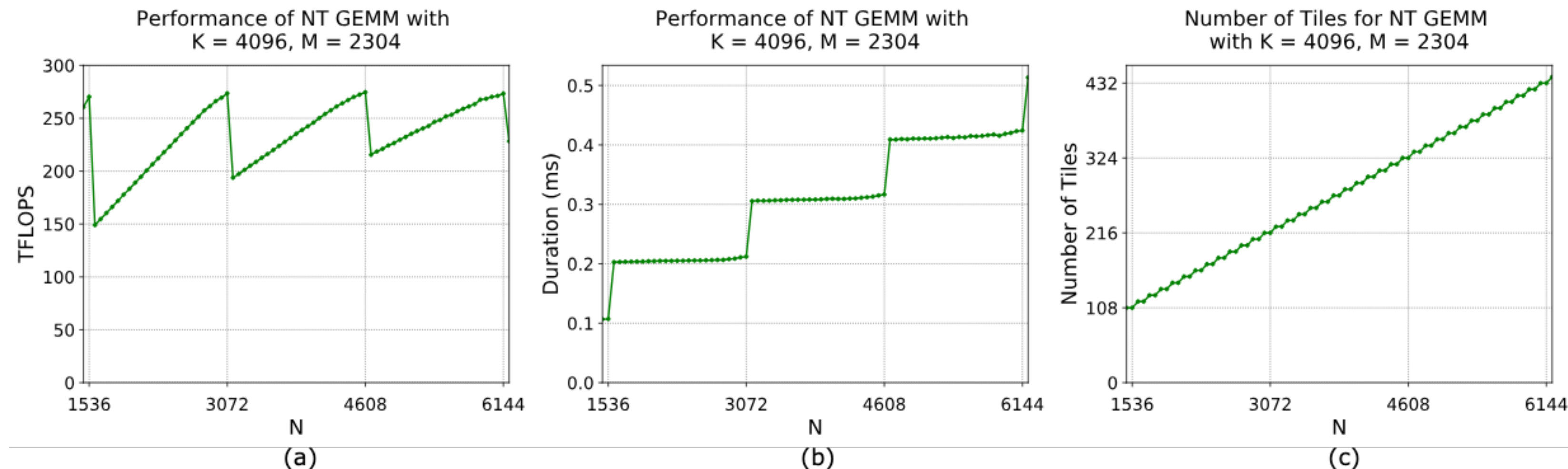
Inconsistent input shapes

- **Wave quantization** — total number of tiles (thread blocks) is quantized to the number of SMs.
- Kernel will be executed in waves.
- Number of waves is $\left\lceil \frac{\text{Tiles}}{\text{Num. of SMs}} \right\rceil$
- The last wave can underutilize the GPU
- Best utilization occurs with the number of tiles is the multiple of SMs



Inconsistent input shapes

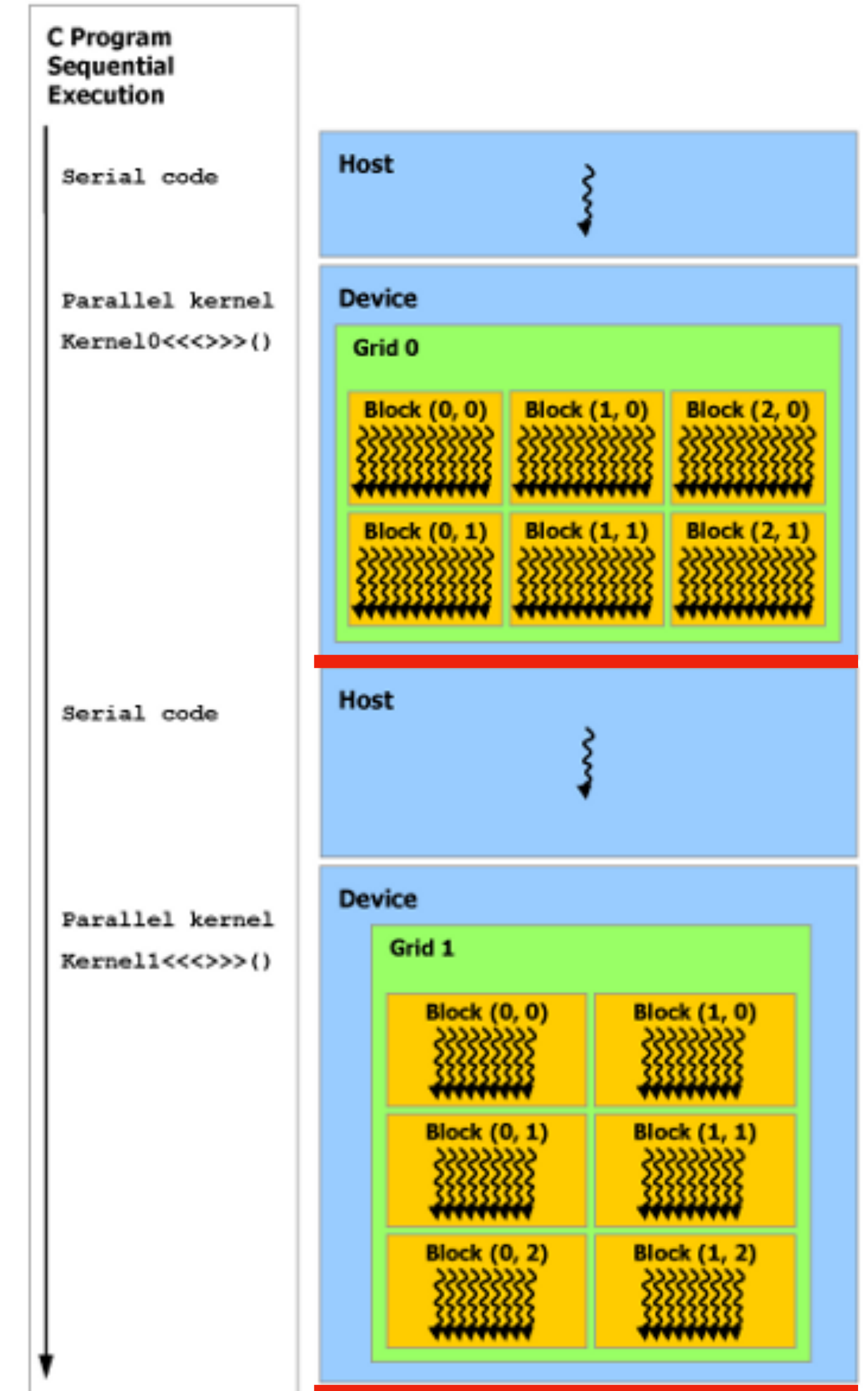
- **Wave quantization** — total number of tiles (thread blocks) is quantized to the number of SMs.
- The last wave can underutilize the GPU



The effects of wave quantization in terms of (a) achieved FLOPS throughput and (b) elapsed time, as well as (c) the number of tiles created. Measured with a function that uses 256x128 tiles over the $M \times N$ output matrix.

Host-device synchronization

- GPU code is executed asynchronously with Python's CPU code
- Any unnecessary CPU-GPU synchronization slows down the GPU execution

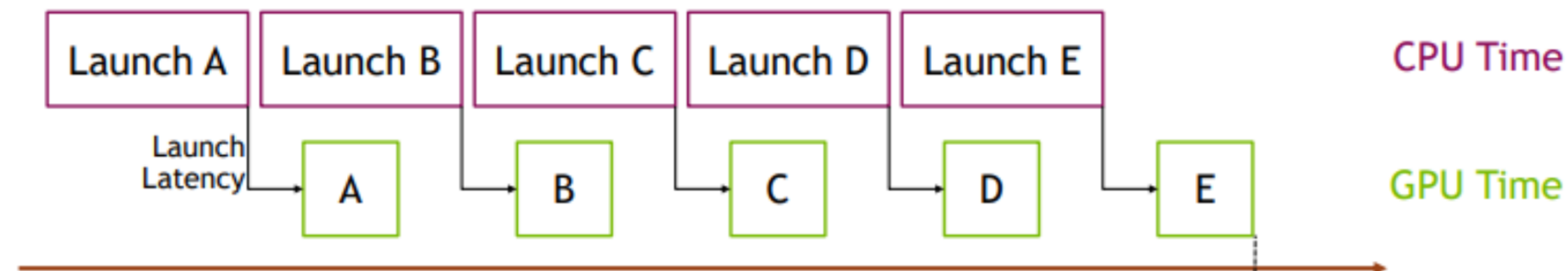


Host-device synchronization

- GPU code is executed asynchronously with Python's CPU code
- Any unnecessary CPU-GPU synchronization slows down the GPU execution
- E.g. `loss.item()` in PyTorch triggers **synchronization**— use `loss.detach()`

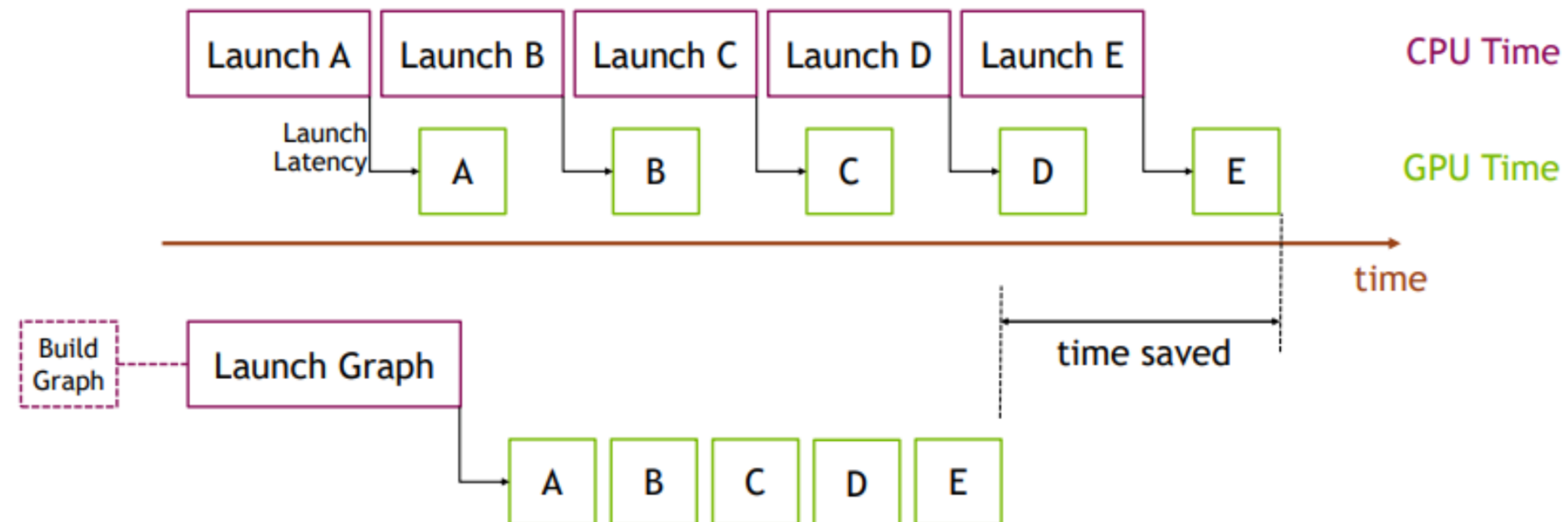
Kernel launch latency

- CPU needs to submit each kernel to the GPU
- Every launch has non-zero fixed overhead
- For large kernels, launch overhead is usually amortized
- For many small kernels, launch overhead can become a bottleneck



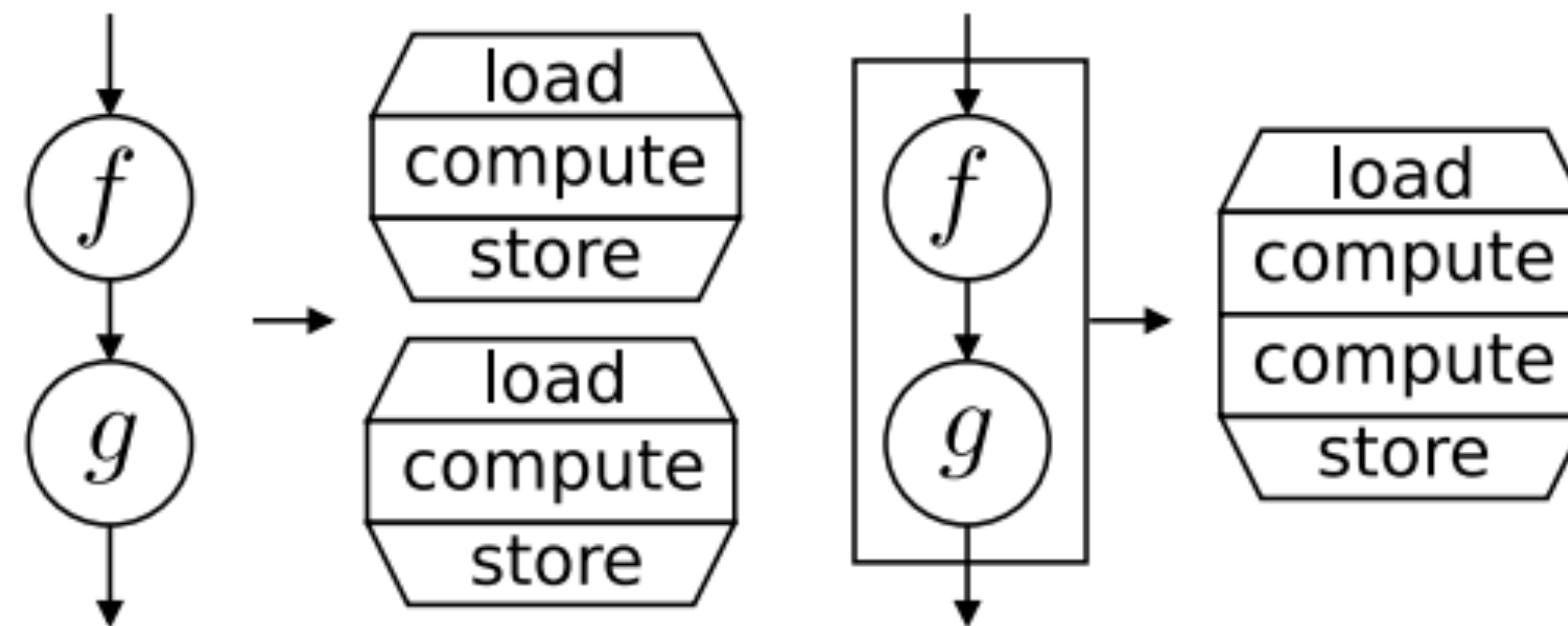
Kernel launch latency - CUDA Graphs

- CUDA Graphs capture a sequence of GPU operations and save execution graph
- Then graph can be relaunched with much less CPU overhead



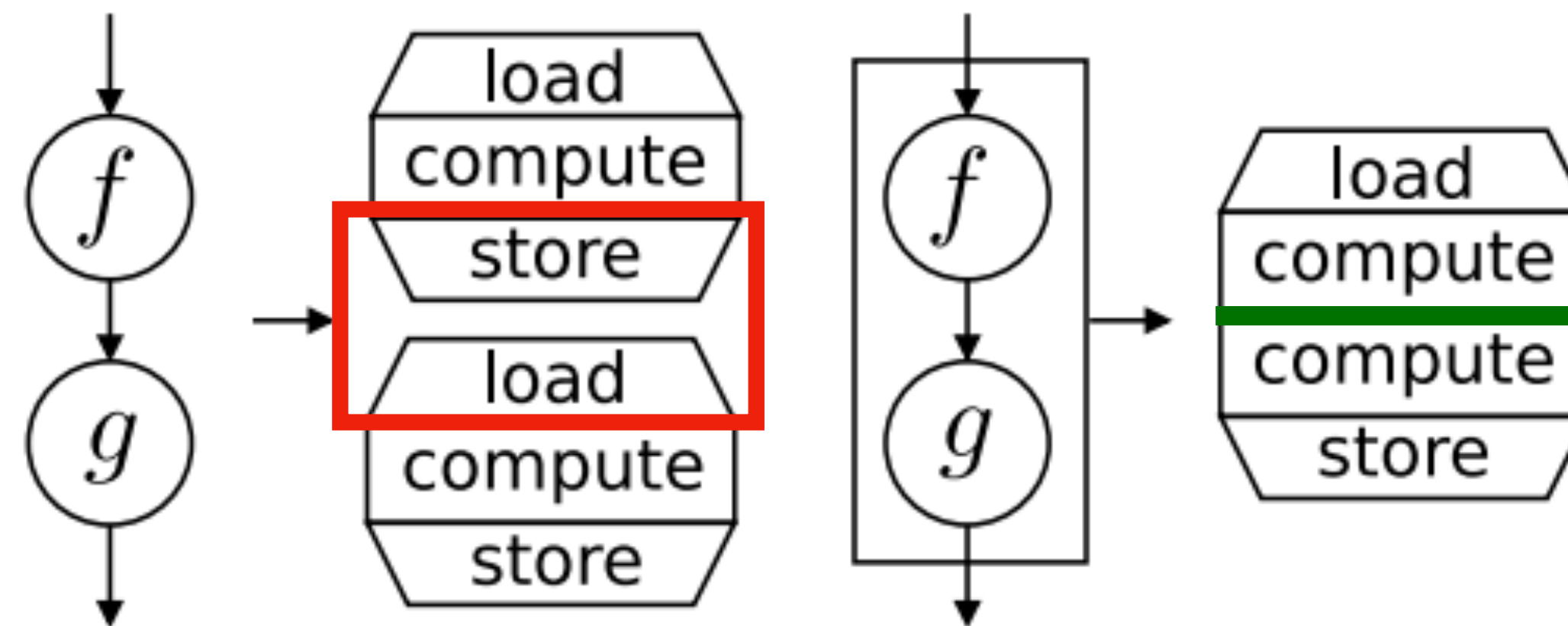
Unfused operations & Kernel fusion

- Typical kernel workflow — load from global memory → execute operation → store in global memory
- For sequential kernel execution, there will be redundant IO-operations — we can place all operations in a single kernel - **fuse** them:



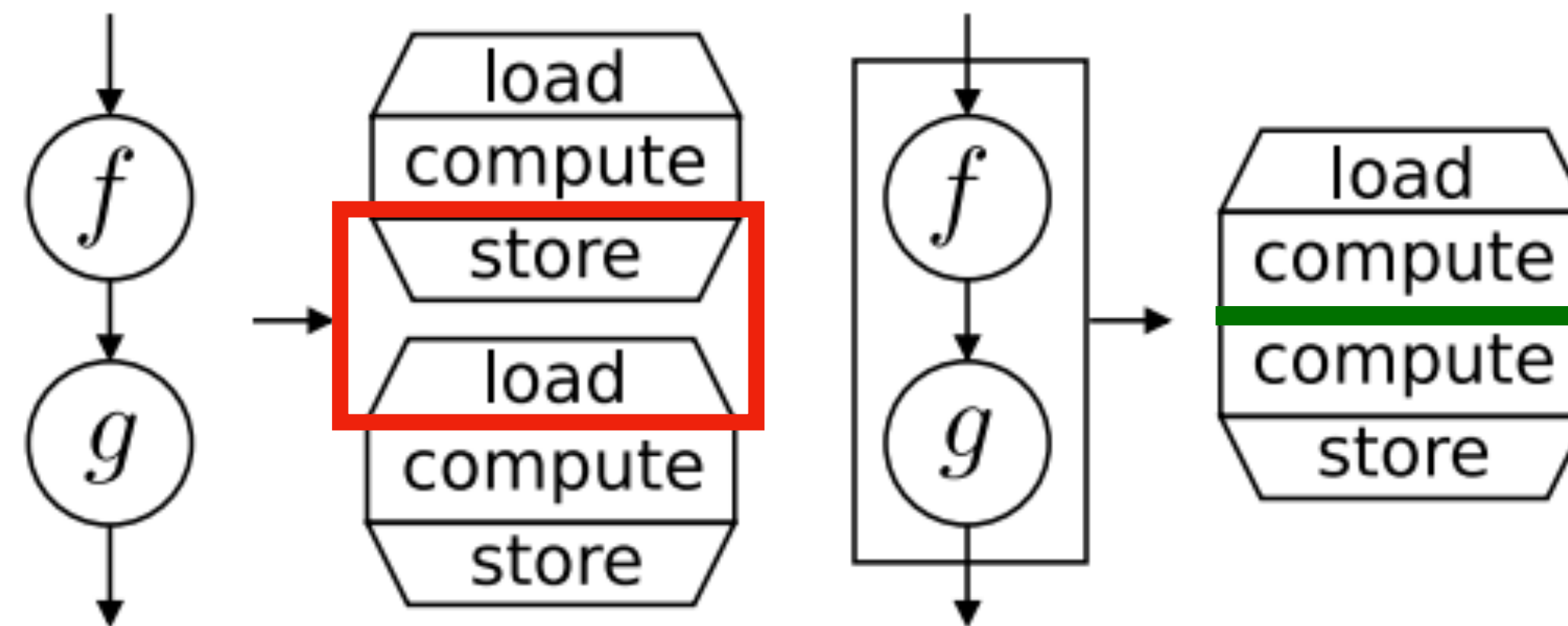
Unfused operations & Kernel fusion

- Typical kernel workflow — load from global memory → execute operation → store in global memory
- For sequential kernel execution, there will be redundant IO-operations — we can place all operations in a single kernel - **fuse** them:



Unfused operations & Kernel fusion

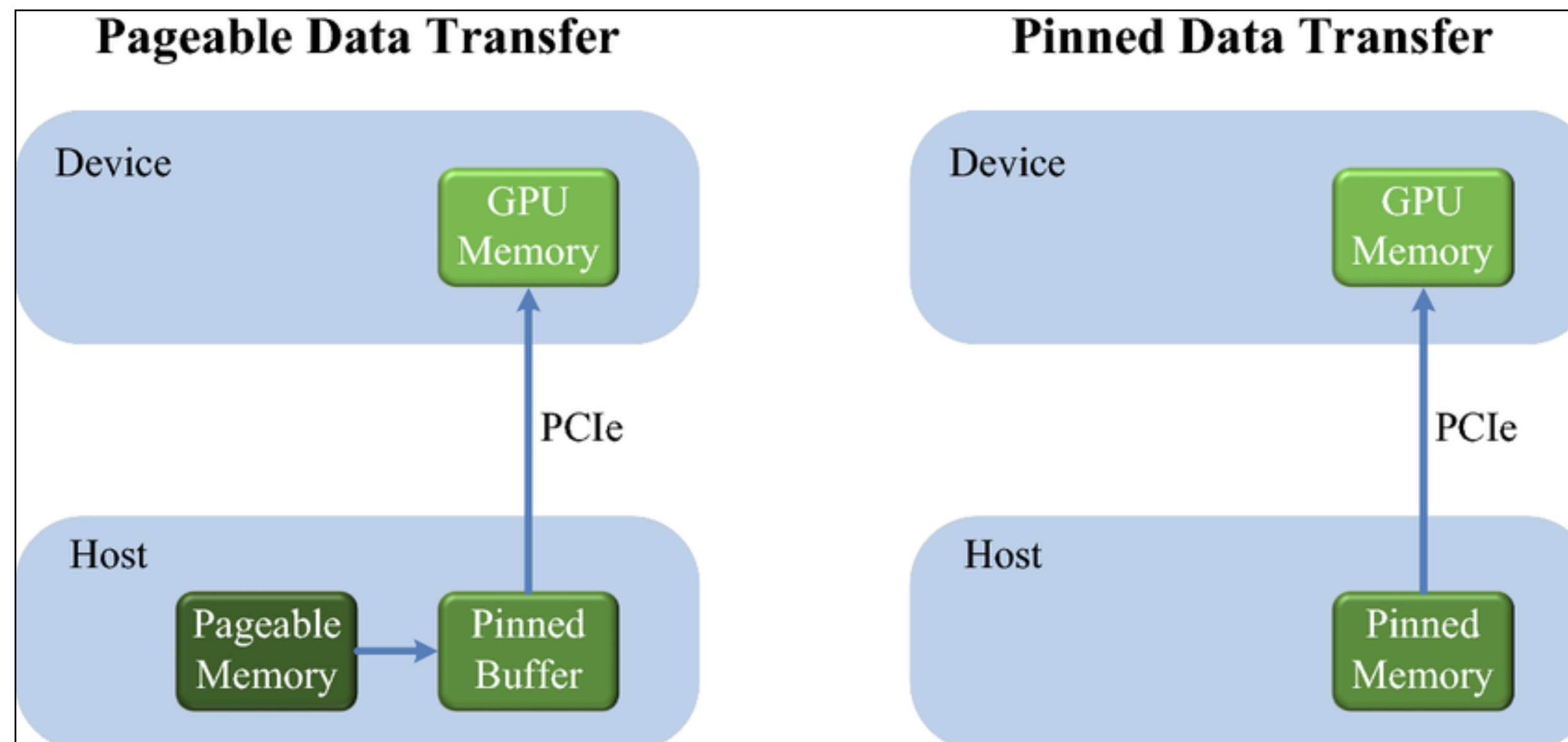
- Typical kernel workflow — load from global memory → execute operation → store in global memory
- For sequential kernel execution, there will be redundant IO-operations — we can place all operations in a single kernel - **fuse** them:



Less memory traffic → higher effective arithmetic intensity

Data transfer

- Host-to-Device (CPU-to-GPU) operations require an object to be located in RAM in a pinned buffer
- Operation system pages RAM and can offload some pages - accessing offloaded pages requires load them to a pinned buffer
- We can pin the memory to speed up the transfer



Identify & deal with bottlenecks
... and speed up the execution

Things you can do:

- Analyze your custom operation via Roofline Model paradigm
- Use proper batch size
- Use specialized hardware primitives
- Improve data locality
- Expose parallelism in kernels design
- Reduce dimensions inconsistencies (e.g. for hidden dims)
- Reduce the amount of CPU-GPU synchronizations
- Use pageable memory ([+ cuda Streams](#) - link)
- Compile model (be careful in distributed scenarios)
- Moving additional operations on GPU might help (e.g. normalizations, data transformations & augmentations)
- **Code profiling** — identify slow code samples (e.g. inefficient forward implementation, slow data transfer)

Where do we express these optimizations?

PyTorch / torch.compile

Triton / TileLang / CuTe DSL

CUTLASS / CUDA C++

PTX

SASS / hardware

Use the highest level that gives you enough control over the bottleneck you need to fix

Code profiling

- Identify slow patterns in the code
- For CPU profiler - `py-spy`
- PyTorch has its own profiler — [torch.profiler.profiler](#)
 - Very handy with Tensorboard

Code profiling

- Identify slow patterns in the code
- For CPU profiler - `py-spy`
- PyTorch has its own profiler — [torch.profiler.profiler](https://pytorch.org/docs/stable/profiler.html)
 - Very handy with Tensorboard
- Also you can try `torch.utils.bottleneck`
- For distributed job analysis, you can use Holistic Trace Analysis — https://docs.pytorch.org/tutorials/beginner/hta_intro_tutorial.html

torch.compile

- Just-in-Time model compilation
- Inspects PyTorch code & modules, makes graph representation
- Optimizes code by making optimized kernels (e.g. fused operators) & removing python overhead
- Different shapes of the input tensors can break fused operations, PyTorch will need to recompile a model
- Use with caution! (see more on a seminar)